Digital Forensics — Lab project report

Forensic Examination of an Android 11 Smartphone (Google Pixel 3)

Integrity verification, automated analysis with ALEAPP, manual SQLite verification with DB Browser, and second-tool processing with Autopsy

Student	Naoual Wafiq
Course	Digital Forensics — Prof. S. Lazaar — ENSA Tangier
Date	4 November 2026
Evidence	“Android 11 – Pixel 3 – Data.tar” (Digital Corpora), 17,205,267,456 bytes SHA-256 d4cb326a41cca5cc23a2c2cb1908498c7281b1b2fcc0f8781fb6b3945e6c815c
Status	Final version

How to read this report

Three kinds of “file” are mentioned. (A) **Files on our disk** — the archive we downloaded and our working folders (e.g. `D:\Enquete_Android11\01_Original\android_11.zip`). (B) **Files inside the evidence** — files that were on the phone, written as paths inside the image (e.g. `data/data/com.whatsapp/databases/msgstore.db`). (C) **Files produced by our tools** — the ALEAPP report, the Autopsy case, our hash lists. Section 2 explains where each kind is.

Times are in **UTC** unless written “local”. The phone was set to **UTC−4** (US Eastern Daylight Time), proven in section 5.2.

Every finding states its source: the file inside the phone, the tool, and the figure. Sensitive values (passwords, tokens, Wi-Fi keys, street addresses) are **[REDACTED]**; GPS coordinates are rounded to two decimals (about 1 km).

Contents

Section	Title
—	Project at a glance (diagram of the whole work)
1	Introduction: case, scenario and investigation questions
2	The evidence: what we downloaded and what it contains
3	Method, tools and environment
4	Preservation: hashes, comparisons and chain of custody
5	Automated analysis with ALEAPP — findings by phase
6	Manual verification with DB Browser for SQLite (slide 33)
7	Cross-validation, automated verification and agents
8	Second tool: Autopsy (Windows) — processing and results
9	Consolidated timeline
10	Conclusions, answers to the questions and limitations
A	Glossary

Project at a glance

The diagram summarises the whole project, from the phone in 2020 to this report. Grey: what the image author did. Blue: my investigation. Green: the result. Details are in the sections indicated in the Contents.

2020 — WORK OF THE IMAGE AUTHOR (Joshua Hickman)		
PHONE	**Google Pixel 3, Android 11.** 11 Sep 2020: factory reset (22:59 UTC), rooted with Magisk 12 min later. Then used for 3.5 weeks like a real user: messages, calls, file transfers, vault apps, trips in Holly Springs, NC.	**→ 3.5 weeks of realistic activity**
▼		
ACQUISITION	**5 Oct 2020, 19:47–19:56 UTC.** Magnet ACQUIRE copies the phone through ADB with root → **Data.tar** (17.2 GB, file-system extraction of /data) + **image_info.txt** (MD5 and SHA-1 of the tar). Published on Digital Corpora as android_11.zip.	**→ Data.tar + image_info.txt (the evidence)**
2026 — MY INVESTIGATION (examiner who receives the evidence)		
1 · PRESERVE	Downloaded android_11.zip and extracted Data.tar. **MD5 and SHA-1 identical to the 2020 record**, on Mac and on Windows; SHA-256 identical before and after every tool. A USB disk failure produced a truncated copy: **detected by the hash**, redone.	**→ Evidence proven intact**
▼		
2 · ALEAPP	Automated analysis (1,317 modules, 30 min). Device and owner, root and the traces of the acquisition, communications with “This Is DFIR Two”, file transfers, two vault apps, locations, apps — **answers to Q1–Q7**, each traced to its source file.	**→ Findings for Q1–Q7**
▼		
3 · DB BROWSER (slide 33)	Manual SQL on copies of mmssms.db, msgstore.db, accounts_ce.db. **Confirmed** 27 SMS, 2 MMS, 17 accounts. **Found more than ALEAPP:** one deleted MMS (sqlite_sequence), 5 “empty” WhatsApp messages = location shares 34 m apart (“Here I am!” / “Me, too!”). Extracted copies re-hashed: identical.	**→ ALEAPP validated + 2 new findings**
▼		
4 · AUTOPSY	Second tool on Windows: rebuilt the /data partition (incl. magisk_backup), produced call, account and web artifacts — but **no SMS** and tables that did not display: documented as a limitation.	**→ Structure confirmed; limits documented**
▼		
5 · AUTOMATION & AGENTS	**v1 script:** repeats the whole procedure in 2 min, same results. **v2 agent** (observe → decide → act → record, 10 rules): 3,866 files, 328 decisions, **10 encrypted WhatsApp backups decrypted**. **v3 gap explorer:** knows no app, finds what the tool did not read (17,751 files) and proposes where to look.	**→ Repeatable checks; 10 backups decrypted**
RESULT		
REPORT	Answers to the 7 questions, timeline 11 Sep → 5 Oct 2020, limitations. Within the course scenario the **channels** of transfer and concealment are proven; the **confidential nature** of the data is not (test phone).	**→ Answers, timeline, limits**

The thread that runs through every step: integrity and verification

Every step works on **copies**, never on the evidence; every copy is **hashed before and after**; every result is **checked by a second method** (ALEAPP ↔ manual SQL ↔ Autopsy ↔ agent). The chain of custody records who did what, when, with which tool and which hash.

1. Introduction: case, scenario and investigation questions

1.1 Context

Digital forensics is the identification, collection, preservation, examination and analysis of digital data so that the results can be used as evidence (course, slide 3). Smartphones concentrate communications, locations, photos and application data, which makes mobile forensics a central branch of the discipline (slide 19).

1.2 Scenario

A phone has been submitted for forensic examination after concerns about an employee's **private communications** and a possible **leak of confidential files**. The investigator must copy the phone without altering it, **reconstruct the communication activity**, identify the relevant messages, calls and file exchanges, determine whether data was hidden, check whether **deleted messages leave recoverable traces**, and build a timeline (course case study, slides 28–35). The scenario is applied to a public reference image.

1.3 Investigation questions

#	Question
Q1	Which device is it, and who uses it?
Q2	With whom did the user communicate, through which channels, and when?
Q3	Did files enter or leave the device, and through which channels?
Q4	Did the user try to hide data or activity?
Q5	Where was the user?
Q6	What did the user install, search and browse?
Q7	In what order did the key events happen?

1.4 Summary of the work done

- **Preservation:** hashes of the evidence computed on macOS and on Windows and compared with the 2020 acquisition record — identical.
- **Automated analysis:** ALEAPP parsed the image (1,317 modules, 30 min 21 s); the findings are organised by phase in section 5.
- **Manual verification (slide 33):** three databases extracted from the image and queried with SQL in DB Browser for SQLite; results identical to ALEAPP, plus new findings (deletion traces, location messages that ALEAPP showed as empty).
- **Second tool:** Autopsy 4.23.1 on Windows extracted the image and produced call, account and web artifacts, but no SMS (section 8).

2. The evidence: what we downloaded and what it contains

2.1 Origin of the image

The evidence is the public **“Android 11” image** distributed by **Digital Corpora** (digitalcorpora.org), a reference collection of forensic test data. It was created by **Joshua Hickman**, who used a **Google Pixel 3** running **Android 11** like a normal user between September and October 2020 (installing apps, messaging, browsing, travelling), then made a forensic copy on **5 October 2020**. Because it is a test device, the data are realistic but no real person is accused. The author also provides documentation describing what he did (the “answer key”); we deliberately did **not** read it before the analysis so that it can serve as an independent check afterwards.

2.2 Our case folder on the external disk (kind A files)

All evidence and work products are kept on an external 2 TB disk (TOSHIBA, exFAT), seen as `/Volumes/TOSHIBA` on the Mac and as drive `D:` on the Windows PC. The case folder is organised as follows:

```
Enquete_Android11/
├── 01_Original/
│   └── android_11.zip                     archive downloaded from Digital Corpora (never modified)
├── 02_Travail/                            working copies
│   └── Android 11/
│       ├── Android 11 Image with Documentation_ALTEREE.zip   damaged copy, set aside (section 4.4)
│       └── Android 11 Image with Documentation/
│           ├── Extraction/
│           │   └── Android 11 - Pixel 3 - Data.tar     THE EVIDENCE (17.2 GB)
│           ├── Extraction Info/
│           │   ├── image_info.txt          acquisition record (2020)
│           │   └── activity_log.txt        log of the acquisition tool
│           └── (author's documentation of the activity – answer key)
├── 03_Rapports/
│   └── ALEAPP_Output_2026-10-19_Monday_134141/   ALEAPP report (HTML + TSV)
├── 04_Documentation/                      chain-of-custody notes
├── 05_Captures/                           screenshots used as figures
└── 06_Autopsy/                            Autopsy case (Windows)

On the Windows PC (internal disk C:):
C:\Enquete\SQLite\extracted\   databases copied out of the .tar (read-only, never opened)
C:\Enquete\SQLite\working\     working copies opened in DB Browser
C:\Enquete\SQLite\notes\       paths.txt, sqlite_hashes_before.txt, sqlite_hashes_after.txt
```

Files whose names start with `._` (e.g. `._Android 11 - Pixel 3 - Data.tar`, 4,096 bytes) also appear on the disk. They are **AppleDouble** files that macOS writes on exFAT disks to store Finder metadata; they are not part of the evidence (identified by their signature in section 8).

2.3 What kind of copy the .tar is

Slide 30 distinguishes three acquisition levels: **logical** (what the phone agrees to export, e.g. a backup), **file system** (all files, including the private databases of apps) and **physical** (bit-for-bit copy of the flash memory, including free space). Our evidence is a **file-system extraction of the /data partition**, made with **Magnet ACQUIRE** through ADB with root rights and packed in a **.tar** archive (a format that stores files and folders with their paths and dates). Consequence: we have every file of the user partition, but **not** the unallocated space where deleted files could be carved.

2.4 What is inside the .tar (kind B files)

Opening the .tar shows one top folder, `data/`: the phone's **/data partition**, where Android keeps all user data. The first bytes of the .tar are literally `data/` (Figure in section 8). Its main sub-folders are:

Folder in the image	What it contains	Encryption class (FBE)
data/data/<package>/	Private data of each app (databases, preferences). Example: data/data/com.whatsapp/databases/msgstore.db	CE
data/user/0/<package>/	Same content as data/data — on Android /data/user/0 is a **symbolic link** to /data/data (proved by identical hashes, section 6.3)	CE
data/user_de/0/<package>/	App data needed before the PIN is entered (telephony, root manager, alarms)	DE
data/system_ce/0/	System data of user 0 protected by the PIN (accounts_ce.db, usage statistics)	CE
data/system_de/0/	System data readable at boot (accounts_de.db)	DE
data/system/users/	List of users and profiles (userlist.xml, 0.xml, 10.xml)	—
data/misc/adb/	Computers authorised to use ADB (adb_keys, adb_temp_keys.xml)	—
data/misc/bootstat/	Boot statistics, including the factory_reset record	—
data/media/0/	Internal shared storage, seen by the user as /storage/emulated/0 (Download, Pictures, DCIM, WhatsApp, hidden vault folders)	CE

CE (Credential Encrypted) storage can only be decrypted after the user has unlocked the phone with the PIN; **DE** (Device Encrypted) storage is available from boot. This is Android 11's **File-Based Encryption**. Because our copy contains CE data, the phone was unlocked during the acquisition.

2.5 Key files of the phone cited in this report

File inside the image	What it is	Used in
data/system_ce/0/usagestats/version	Android version and build	5.2
data/user_de/0/com.android.providers.telephony/databases/telephony.db	SIM cards (number, IMSI, ICCID, carrier)	5.2
data/system_ce/0/accounts_ce.db	Accounts configured on the phone (17)	5.2, 6.7
data/system_de/0/accounts_de.db	Debug log of account additions/removals (local time)	5.2
data/system/users/userlist.xml, 0.xml, 10.xml	Users and profiles (owner + guest)	5.2
data/misc/bootstat/factory_reset	Time of the factory reset	5.3
data/user_de/0/com.topjohnwu.magisk/databases/sulogs.db	Magisk root request log	5.3
data/misc/adb/adb_temp_keys.xml	Computers authorised through ADB	5.3
data/data/com.android.providers.telephony/databases/mmssms.db	SMS and MMS (active database)	5.4, 6.4–6.5
data/user_de/0/com.android.providers.telephony/databases/mmssms.db	Second SMS database — empty	6.4
data/user_de/0/com.android.providers.telephony/app_parts/	Image files of the MMS	6.5
data/data/com.android.providers.contacts/databases/contacts2.db	Contacts	5.4
data/data/com.android.providers.contacts/databases/calllog.db	Call log	5.4
data/data/com.whatsapp/databases/msgstore.db (+ -wal, -shm)	WhatsApp messages	5.4, 6.6
data/data/com.android.providers.downloads/databases/downloads.db	Download history (Nearby Share, Tutanota…)	5.5
data/media/0/.privacy_safe/db/privacy_safe.db	HLD Vault hidden-file database	5.6

3. Method, tools and environment

3.1 Forensic process

The work follows NIST SP 800-86 (Collection → Examination → Analysis → Reporting, slide 12) and ISO/IEC 27037 (identification, collection, acquisition, preservation, slide 27), applied to the smartphone case study of slides 28–35.

Course step	Standard	What was done
Seizure, isolation (slide 29)	ISO 27037: identification, collection	Done by the image author in 2020; device identity documented from the acquisition record.
Forensic copy (slides 30–32)	ISO 27037: acquisition	Done by the author with Magnet ACQUIRE (file-system extraction). We verified its provenance and integrity.
Preservation	ISO 27037: preservation	MD5, SHA-1 and SHA-256 at every step; original archive never modified; working copies only; read-only flags.
Recover messages and deleted rows (slide 33)	NIST: examination	ALEAPP, then manual SQL queries in DB Browser for SQLite on mmssms.db, msgstore.db, accounts_ce.db.
Application artifacts (slide 34)	NIST: examination	ALEAPP (1,317 modules); Autopsy 4.23.1 as a second tool.
Timeline and report (slide 35)	NIST: analysis, reporting	Correlation of sources, UTC/local conversion, this report.

3.2 Tools and environment

Tool	Version / platform	Purpose
md5, shasum (Terminal)	macOS 15.0.1 (24A348)	Hashes on the Mac
Get-FileHash (PowerShell)	Windows PC	Hashes on Windows
unzip, tar	built into macOS and Windows	Extraction of the archive; listing and extraction of selected files from the .tar
ALEAPP	2026.4.4-dev, 1,317 modules, Python 3.12.15 (macOS)	Automated parsing of Android artifacts
DB Browser for SQLite	Windows, 64-bit standard installer	Manual examination of SQLite databases (read-only)
Autopsy	4.23.1 (Windows)	Second-tool processing (extraction, file typing, Android parsers)
diskutil	macOS 15.0.1	Verification and repair of the external disk

3.3 Conventions

- Times in UTC unless written “local” (UTC−4).
- Facts (what a file contains) are separated from interpretations (what it probably means), which are marked as such.
- Sensitive values are [REDACTED]; coordinates rounded to two decimals.

4. Preservation: hashes, comparisons and chain of custody

4.1 What a hash proves

A hash function computes a fixed-length **fingerprint** of a file. The same file always gives the same value; changing a single bit gives a completely different value. Comparing two hashes of the same file taken at two moments therefore proves whether the file changed between those moments. MD5 (128 bits, 32 hexadecimal characters) and SHA-1 (160 bits, 40 characters) have known collision weaknesses; SHA-256 (256 bits, 64 characters) is the current standard. Upper- or lower-case letters do not matter: `ACA33F3D…` and `aca33f3d…` are the same number.

4.2 The acquisition record written in 2020 (image_info.txt)

The folder `Extraction Info/` of the archive contains `image_info.txt`, written automatically by the acquisition tool when the copy was made. It is the **reference** for integrity:

Field	Value	Meaning
Imaging tool	Magnet ACQUIRE v2.30.0.22097, on Windows 64-bit	Free acquisition tool from Magnet Forensics
Imaging start / end	2020-10-05 19:47:57 → 19:56:41 UTC	The copy took about 9 minutes
Device	Google Pixel 3 (blueline), Android 11, build RP1A.200720.009	Identifies the phone
Serial number	8CEX1N716	Unique device identifier (slide 29)
Encryption	encrypted, type: file (FBE)	File-Based Encryption
Segment MD5	ACA33F3DE4E228C29BC39576865FE7D9	**Reference hash** of the .tar
Segment SHA-1	A7A4B184946AF72C97395462816931C453F073C4	**Second reference hash**
Examiner / evidence number / description	(empty)	Documentation gap in the original record

The second file, `activity_log.txt`, is the step-by-step log of the acquisition tool (connection, checks, copying, hashing). Its MD5 (`26a042518e6d4c41ab0eadeca2b899b4`) matches the recorded value.

4.3 The comparisons we made

Each comparison below compares **two hashes of the same file** at two moments (or by two parties). “Identical” means the file did not change between those moments.

Comparison 1 — 2020 record versus our calculations (the most important)

	Who / when / how	MD5	SHA-1
A. Reference	Magnet ACQUIRE, 2020-10-05, written in image_info.txt	ACA33F3DE4E228C29BC39576 865FE7D9	A7A4B184946AF72C97395462 816931C453F073C4
B. Mac	Us, 2026-10-19, Terminal: md5, shasum -a 1	aca33f3de4e228c29bc3957686 5fe7d9	a7a4b184946af72c973954628 16931c453f073c4
C. Windows	Us, 2026-10-22, PowerShell: Get-FileHash -Algorithm MD5	ACA33F3DE4E228C29BC39576 865FE7D9	—
Result		**A = B = C**	**A = B**

Conclusion: the .tar we analysed is bit-for-bit the file produced by the acquisition tool in 2020, after publication, download, two decompressions, storage on the external disk and use on two computers. MD5 and SHA-1 were computed because those are the algorithms in the 2020 record; SHA-256 was added as a stronger reference.

Comparison 2 — before versus after our tools

File	Moment	SHA-256	Result
Data.tar	Before ALEAPP (Mac, 19/10)	d4cb326a41cca5cc23a2c2cb1908498c7281b1b 2fcc0f8781fb6b3945e6c815c	Reference
Data.tar	After ALEAPP (Mac, 19/10)	d4cb326a…6c815c	Identical — ALEAPP only read the file
Data.tar	Windows, before Autopsy (22/10)	D4CB326A…6C815C	Identical

Comparison 3 — the downloaded archive over time

File	Hash	Result
android_11.zip (10,944,907,975 bytes)	SHA-256 abb27d4c5a57692d192a7619d179749abbd916d9 85d9fb4b10455d96d34d44a7 MD5 9553729d10bc6cae84916a506cb74d98	Identical after transfer to the disk and after the disk repair

Comparison 4 — when hashes did NOT match (detected corruption)

File	Expected	Obtained	Decision
Data.tar, first extraction	17,205,267,456 bytes; MD5 aca33f3d…	1,050,673,152 bytes; different hash	Truncated copy deleted; extraction redone
Inner archive (working copy)	SHA-256 5f3a5118…5fc1	SHA-256 f367c104…f724	Altered; renamed _ALTEREE and not used

Comparison 5 — the SQLite work (details in section 6)

What is compared	Result	Meaning
22 extracted database files, before vs after the manual analysis	IDENTICAL	The extracted copies never changed
Extracted copies vs working copies opened in DB Browser	1 file differs: msgstore.db-shm	SQLite rewrote the WAL index even in read-only mode — reason to use copies
data/data/… vs data/user/0/…	Same hashes	Same files under two paths (symbolic link)
mmssms.db in data/data vs in user_de	Different	Two different databases (active vs empty)

4.4 Incident: disconnection of the external disk

> **Incident report**
> During the first extraction on the Mac, the external exFAT disk disconnected (“Device not configured”). The .tar was truncated (1,050,673,152 of 17,205,267,456 bytes) and the working copy of the inner archive was altered. The volume was repaired (`diskutil repairVolume`, exit code 0), the truncated file deleted and the extraction redone from the intact original, after which all hashes matched. **The hash comparison is what detected the problem**, before any analysis.

4.5 Verification on Windows (actual commands and output)

Before use on Windows, the .tar was marked read-only (a software write protection) and hashed:

```
PS> $TAR = "D:\Enquete_Android11\02_Travail\Android 11\Android 11 Image with
Documentation\Extraction\Android 11 - Pixel 3 - Data.tar"
PS> Set-ItemProperty $TAR -Name IsReadOnly -Value $true
PS> (Get-Item $TAR).IsReadOnly
True
PS> Get-FileHash $TAR -Algorithm MD5
MD5     ACA33F3DE4E228C29BC39576865FE7D9
PS> Get-FileHash $TAR -Algorithm SHA256
SHA256  D4CB326A41CCA5CC23A2C2CB1908498C7281B1B2FCC0F8781FB6B3945E6C815C
```

4.6 Chain of custody

Date (examiner local time)	Action	Tool	Integrity result
18/10/2026	Download of android_11.zip; reference SHA-256 and MD5	Browser; shasum, md5	Reference values recorded
19/10/2026	Extraction of Data.tar (after incident and repair); hashes vs image_info.txt	unzip; md5, shasum	MD5/SHA-1 identical to 2020
19/10/2026	ALEAPP processing (30 min 21 s)	ALEAPP 2026.4.4-dev	SHA-256 identical after
22/10/2026 00:07	Data.tar set read-only and hashed on Windows	PowerShell	MD5 and SHA-256 identical
22/10/2026 00:15–02:39	Autopsy case created; Data.tar added as logical file and processed	Autopsy 4.23.1	Read-only source
22/10/2026 00:42–01:49	22 database files extracted, hashed, analysed on working copies, re-hashed	tar, PowerShell, DB Browser	Extracted copies IDENTICAL

5. Automated analysis with ALEAPP — findings by phase

5.1 What ALEAPP is and how it was used

ALEAPP (Android Logs Events And Protobuf Parser) is an open-source tool that knows where hundreds of Android apps and system services store their data. It opens the .tar **read-only**, finds the known files (databases, XML, protobuf), decodes them and produces an HTML report. Command used on the Mac:

```
python aleapp.py -t tar -i ".../Extraction/Android 11 - Pixel 3 - Data.tar" -o ".../03_Rapports"
```

How to read the ALEAPP report: the left menu groups pages by **category** (grey titles, e.g. “SMS & MMS”); each **page** shows a table and, at the top, the line **“… located at:”** which gives the **file inside the phone** that was parsed. This line is how each finding below is traced back to its source file. All ALEAPP times are UTC unless marked “as stored”.

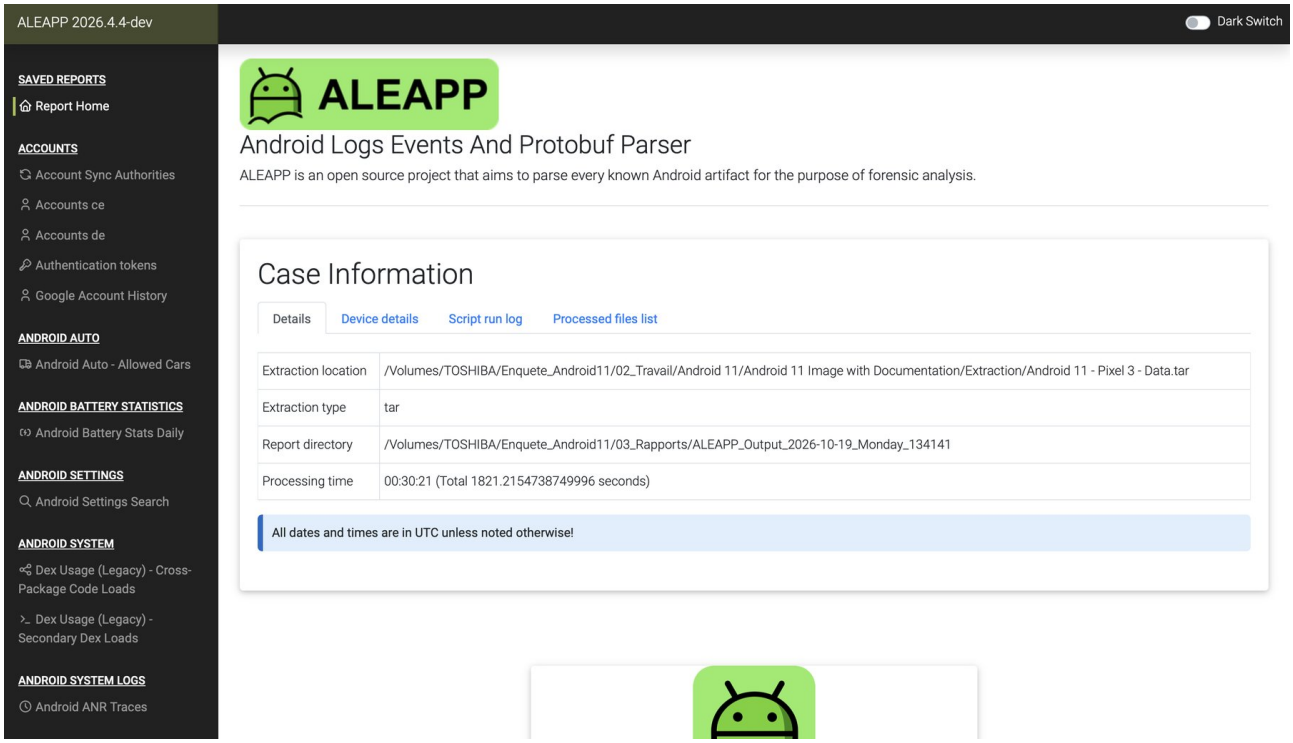


Figure 1. ALEAPP Report Home — Case Information: extraction location (the .tar), extraction type tar, processing time 00:30:21; “All dates and times are in UTC unless noted otherwise”.

5.2 Phase 1 — The device and its owner (Q1)

Finding	Source file in the phone	ALEAPP page
Android 11, build 6720564, codename REL	data/system_ce/0/usagestats/version	Device Information → OS Version
Device name Pixel 3; Bluetooth address 7C:D9:5C:AC:A2:CE; factory reset 2020-09-11 22:59:33; last boot 2020-09-11 23:11:13	several system files	Report Home → Device details
Active SIM: +1 919-579-4674, Google Fi, ICCID 89011203004067132462 (three earlier SIM entries also listed)	data/user_de/0/com.android.providers.telephony/databases/telephony.db	Device Information → siminfo
17 accounts; owner Google account thisisdfir@gmail.com	data/system_ce/0/accounts_ce.db	Accounts → Accounts ce
Owner (user 0, “This Is”) and Guest (user 10) created 2020-10-04 20:55:11	data/system/users/*.xml	Device Users → Android Users and Profiles

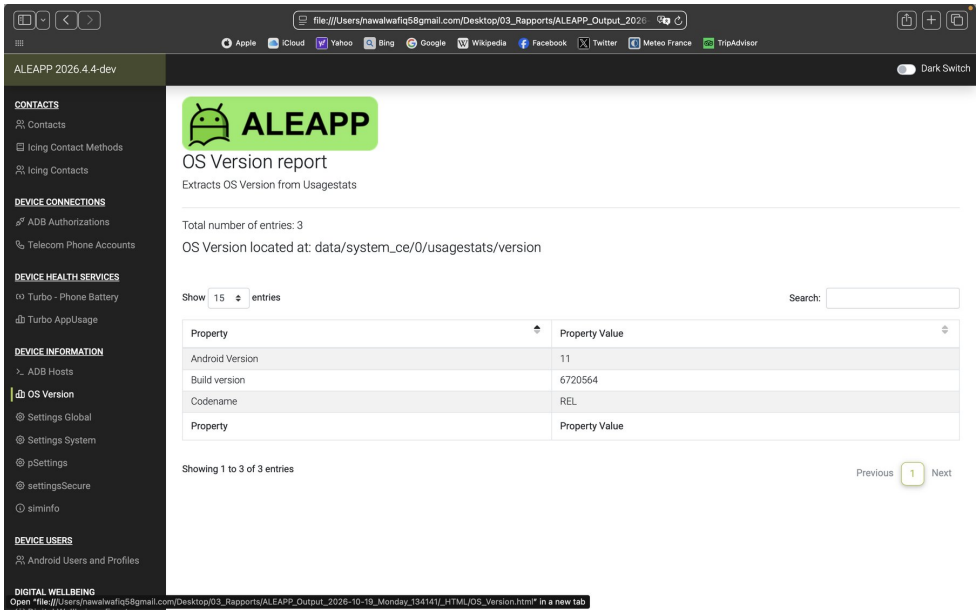


Figure 2. OS Version — Android 11, build 6720564 (source: data/system_ce/0/usagestats/version).

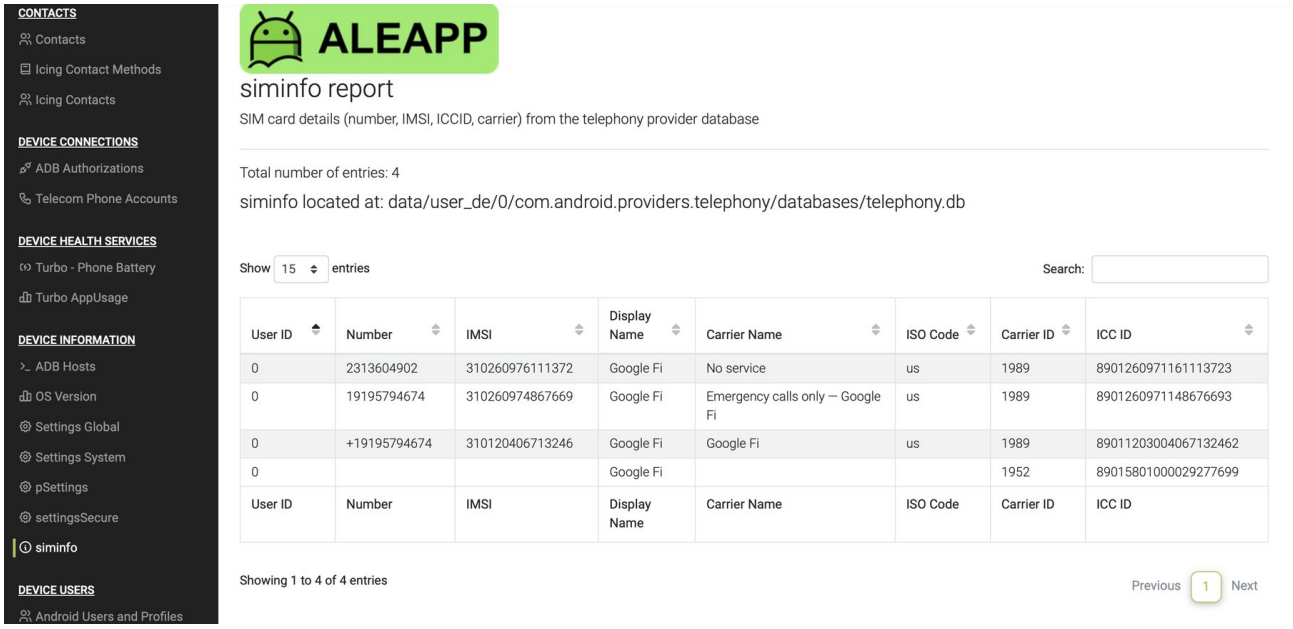


Figure 3. siminfo — SIM entries from telephony.db; the active SIM is +19195794674 (Google Fi, ICCID 8901120300...2462).

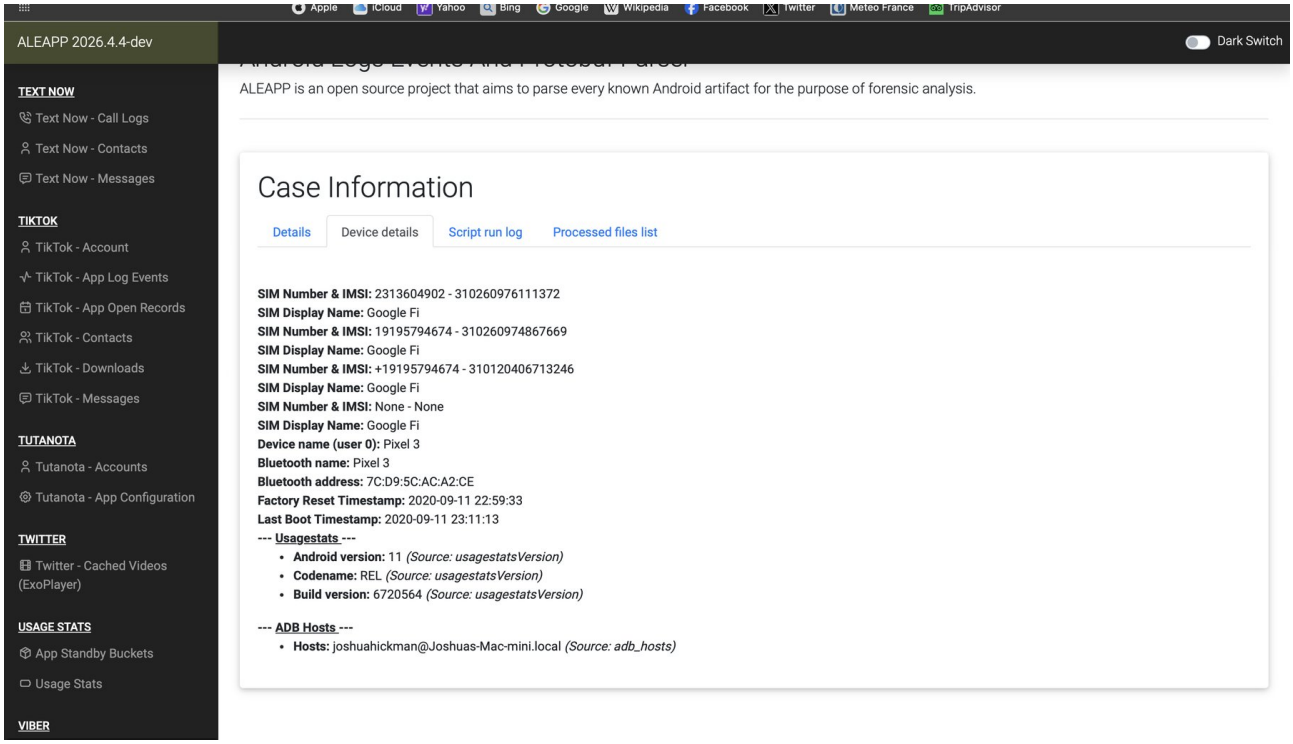


Figure 4. Report Home → Device details — device name Pixel 3, factory-reset and last-boot timestamps, ADB host joshuahickman@Joshuas-Mac-mini.local.

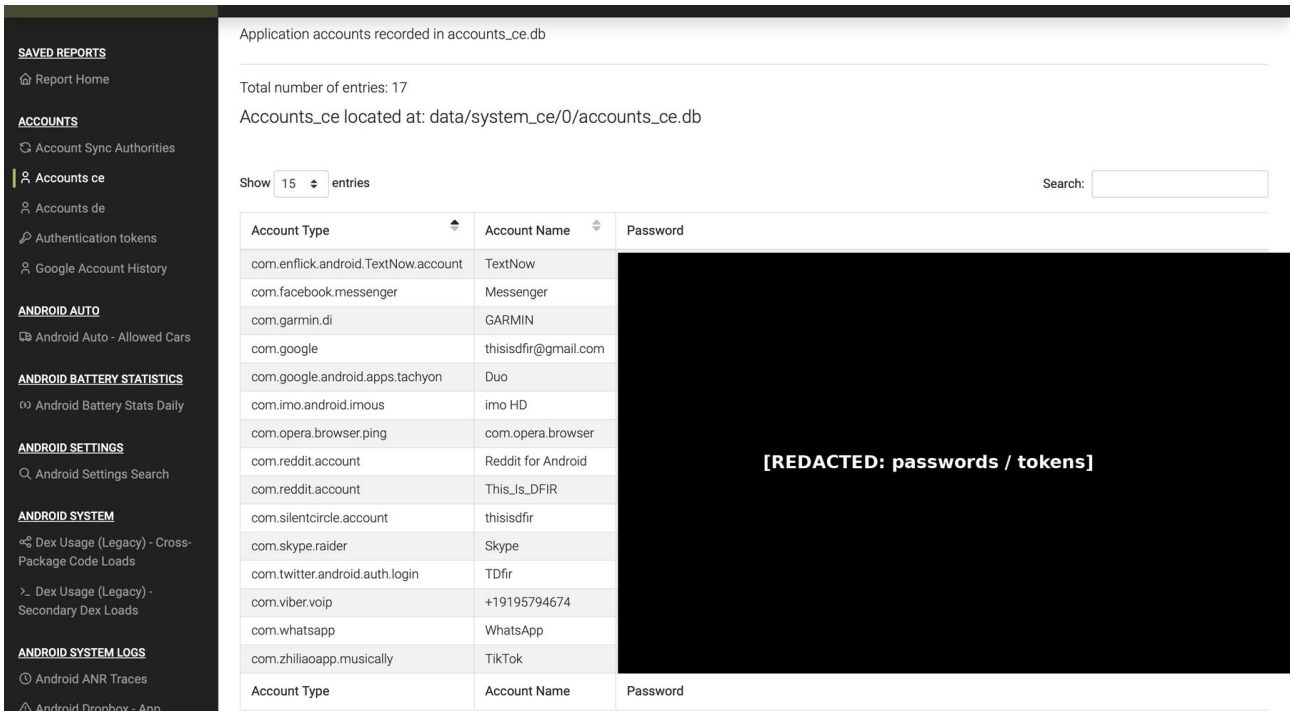


Figure 5. Accounts ce — the 17 accounts in data/system_ce/0/accounts_ce.db. The Password column (tokens and hashes) is [REDACTED].

The device time zone (proof of UTC−4)

accounts_de.db keeps a debug log of account actions whose **Debug Time** is stored in the phone's **local** time, while the **Last password entry** is stored in UTC. For the same account the two values differ by exactly 4 hours:

Account	Last password entry (UTC)	Debug Time (local, as stored)	Offset
thisisdfir@gmail.com	2020-09-11 23:21:39	2020-09-11 19:21:39	−4 h

Account	Last password entry (UTC)	Debug Time (local, as stored)	Offset
Messenger	2020-09-12 12:22:23	2020-09-12 08:22:23	−4 h
imo HD	2020-09-12 20:08:01	2020-09-12 16:08:01	−4 h

The phone was therefore configured in **UTC−4** (US Eastern Daylight Time). The same 4-hour gap appears in HLD Vault (section 5.6) and in the manual SQL conversions (section 6).

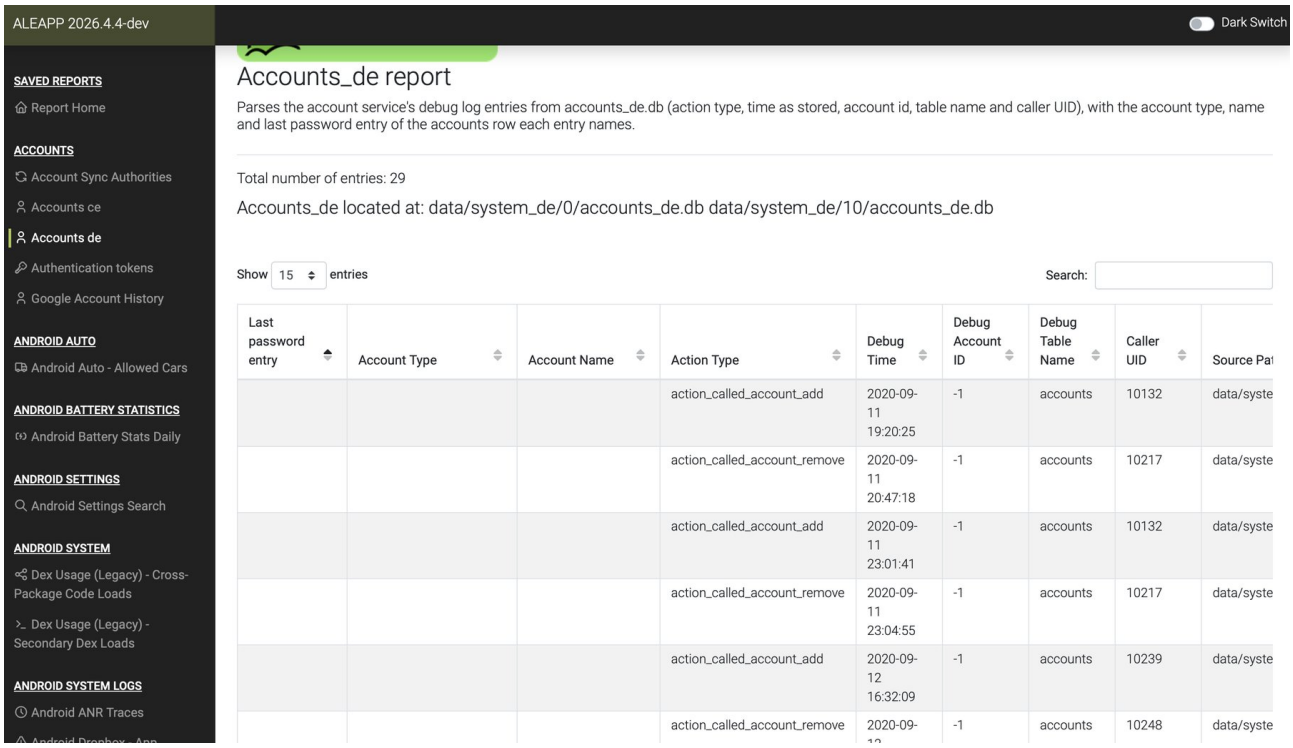


Figure 6. Accounts de — debug log of account additions/removals (data/system_de/0/accounts_de.db); Debug Time is local time.

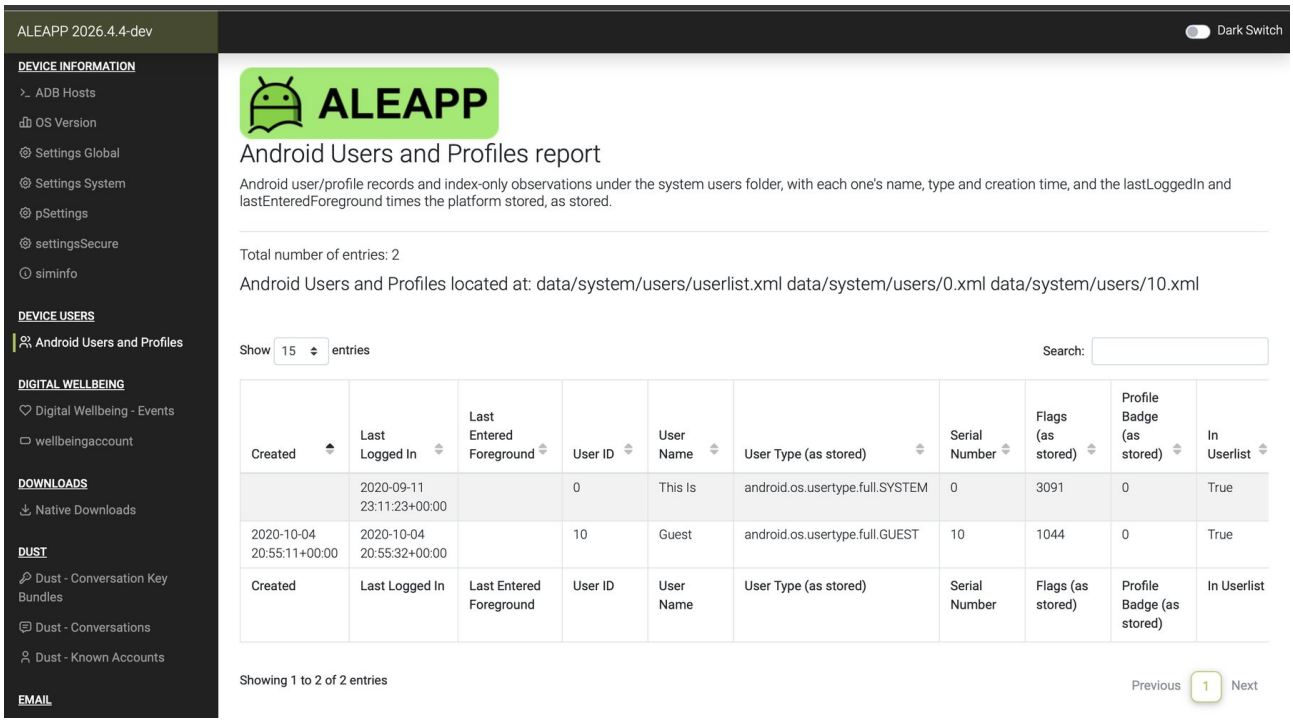


Figure 7. Android Users and Profiles — owner (user 0) and a Guest profile (user 10) created 2020-10-04 20:55:11 and used for 21 seconds.

5.3 Phase 1 (continued) — Reset, rooting and traces of the acquisition

Background. Each Android app runs under its own user number (UID) and cannot read other apps' data. **Root** (UID 0) has every right. **Magisk** obtains root by patching the boot image (boot.img → magisk_patched.img), and records every root request in `sulogs.db`. **ADB** (Android Debug Bridge) lets a computer control the phone over USB; commands run as the user `shell` (package com.android.shell, **UID 2000**). A computer must be approved on the phone, which stores its public key. Root is what allows an acquisition tool to copy /data/data.

Time (UTC)	Event	Source file	ALEAPP page
2020-09-11 22:59:33	Factory reset (first boot after reset)	data/misc/bootstat/factory_reset	Wipe & Setup → Factory Reset
2020-09-11 23:08:31	Magisk app installed	package information	Installed Apps → package info
2020-09-11 23:08:38	boot.img (64 MB) written to Download by **com.android.shell** (pushed from a computer)	MediaStore database	Emulated Storage Metadata - Downloads
2020-09-11 23:09:37	magisk_patched.img written by **com.topjohnwu.magisk**	MediaStore database	Emulated Storage Metadata - Downloads
2020-09-11 23:11:13	Reboot with the patched boot image	boot records	Report Home → Device details
2020-09-11 23:11:48	First root grant: com.android.shell runs /system/bin/sh, UID 2000 → 0	sulogs.db	Magisk → Superuser Log
2020-10-04 09:26:58	ADB key joshuahickman@Joshuas-Mac-mini.local (author's Mac)	data/misc/adb/adb_temp_keys.xml	Device Connections → ADB Authorizations
2020-10-05 19:32:02	ADB key unknown@unknown authorised (acquisition computer)	data/misc/adb/adb_temp_keys.xml	Device Connections → ADB Authorizations
2020-10-05 19:32:46–19:33:43 and 19:47:31–19:48:18	16 root commands: cat /proc/partitions, ls, /data/local/tmp/busybox	sulogs.db	Magisk → Superuser Log
2020-10-05 19:47:57–19:56:41	Imaging	image_info.txt (not in the phone)	—

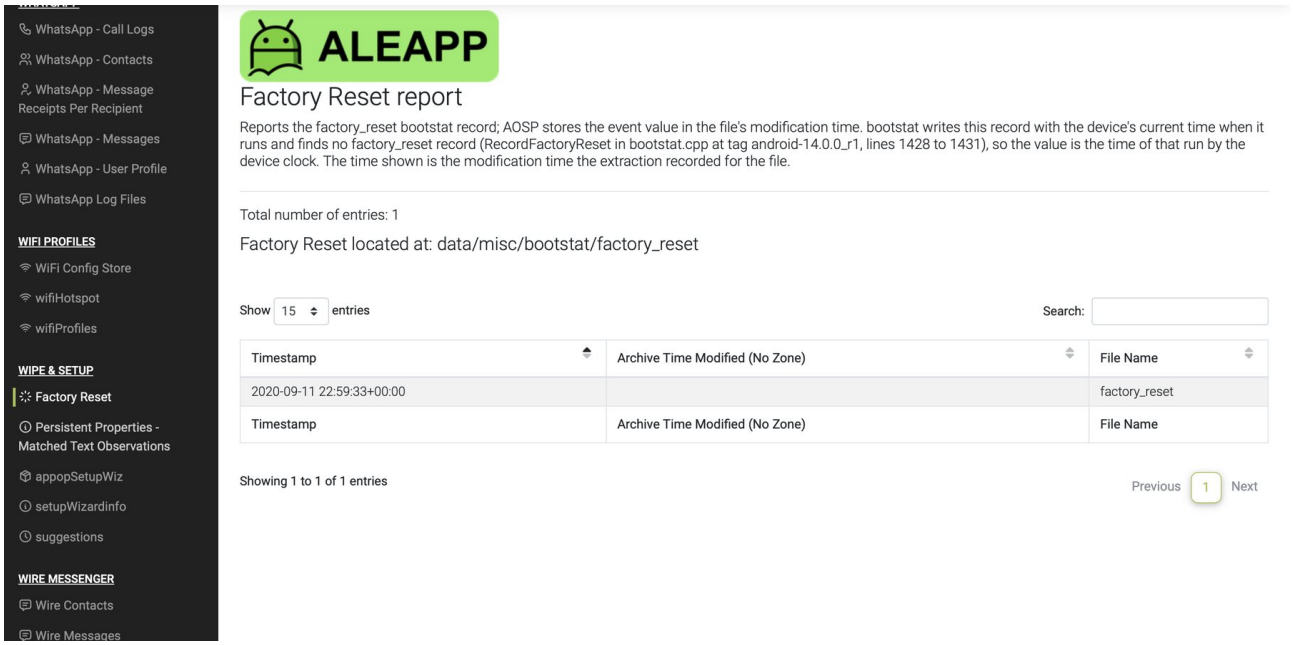


Figure 8. Factory Reset — 2020-09-11 22:59:33 UTC (source: data/misc/bootstat/factory_reset).

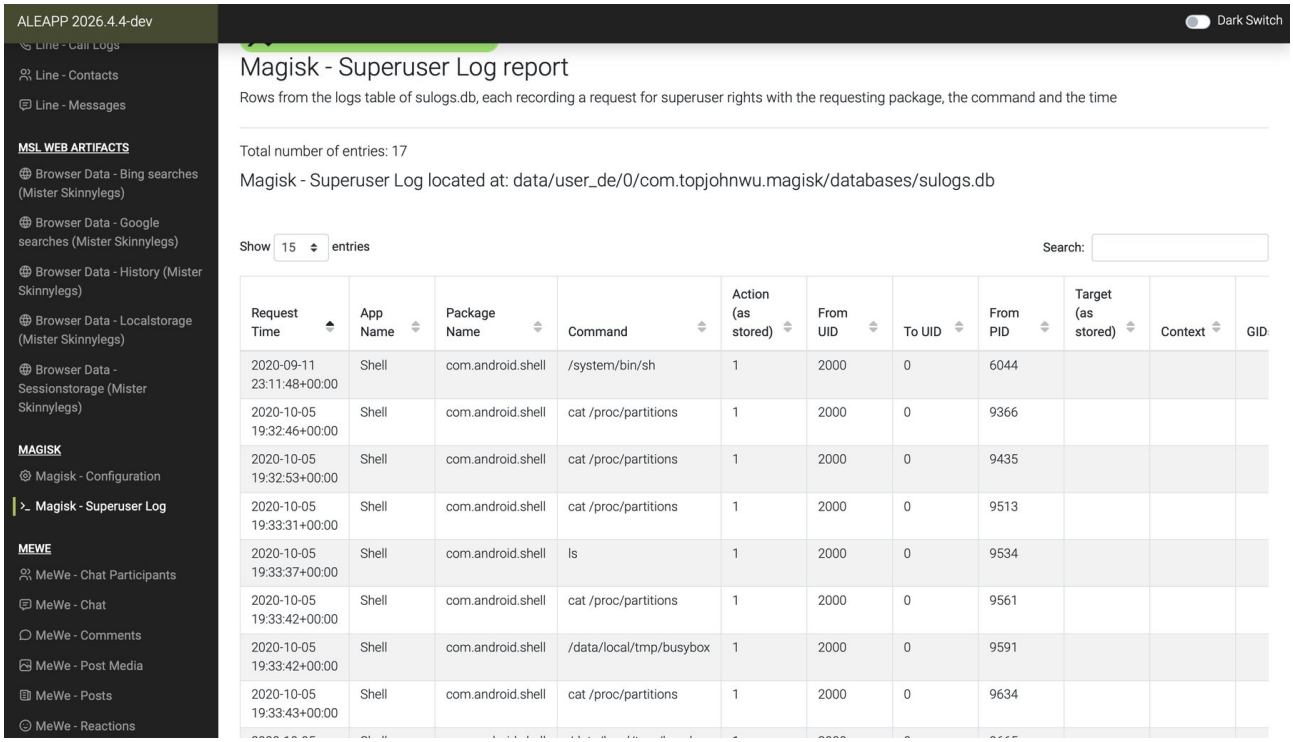


Figure 9. Magisk – Superuser Log (data/user_de/0/com.topjohnwu.magisk/databases/sulogs.db): first root shell on 2020-09-11 23:11:48, then the acquisition tool's commands on 2020-10-05 from 19:32:46 (From UID 2000 → To UID 0).

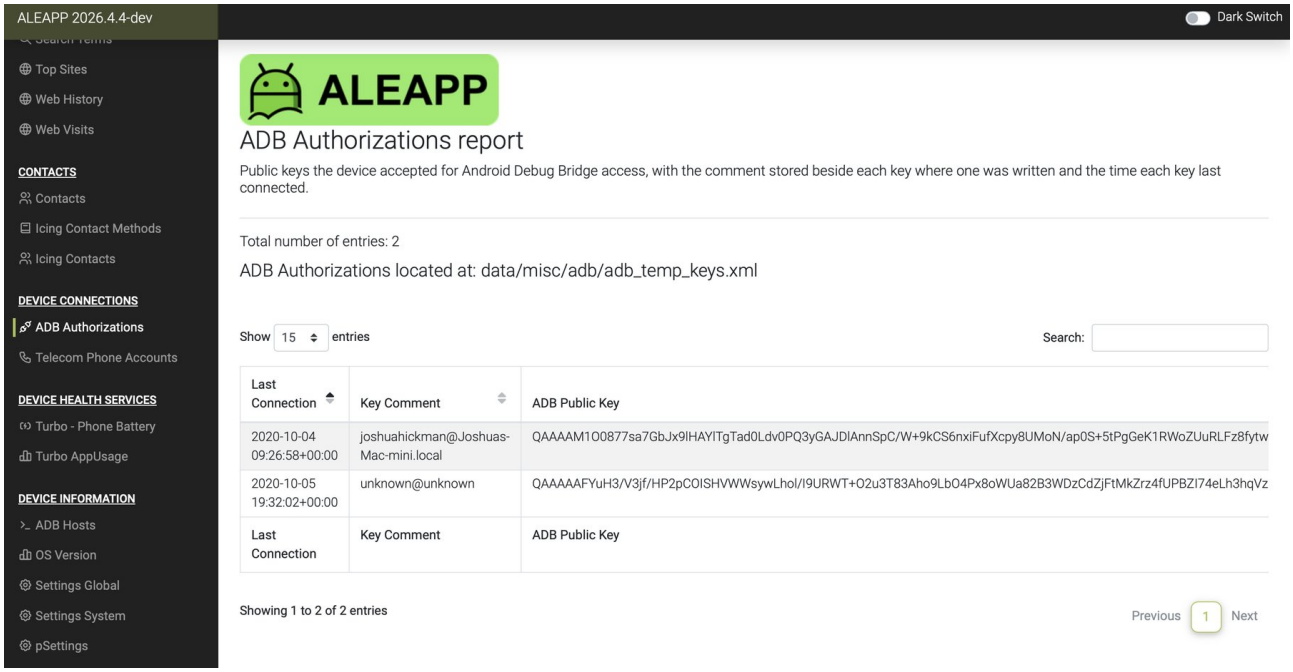


Figure 10. ADB Authorizations (data/misc/adb/adb_temp_keys.xml): the author's Mac mini (2020-10-04) and “unknown@unknown” (2020-10-05 19:32:02).

> **Key point: the phone recorded its own acquisition**
> Three independent sources agree in time and order: the ADB key of the acquisition computer (19:32:02) → the root commands typical of an acquisition tool in sulogs.db (from 19:32:46) → the imaging recorded by Magnet ACQUIRE in image_info.txt (19:47:57). The acquisition therefore left traces in the evidence; every examiner action must be documented so it is not confused with user activity. **Rooting interpretation:** the phone was rooted about 12 minutes after its factory reset, and MagiskHide was enabled (Magisk → Configuration: `magiskhide = true`), which hides root from apps.

5.4 Phase 2 — Communications (Q2)

The main correspondent is **“This Is DFIR Two”**: +1 919-888-7386, thisisdfirtwo@gmail.com (Contacts, source data/data/com.android.providers.contacts/databases/contacts2.db). The same person appears in SMS/MMS, calls, Google Duo, WhatsApp, Wire, Viber, Kik, Telegram, Houseparty, Dust, ProtonMail and Skype.

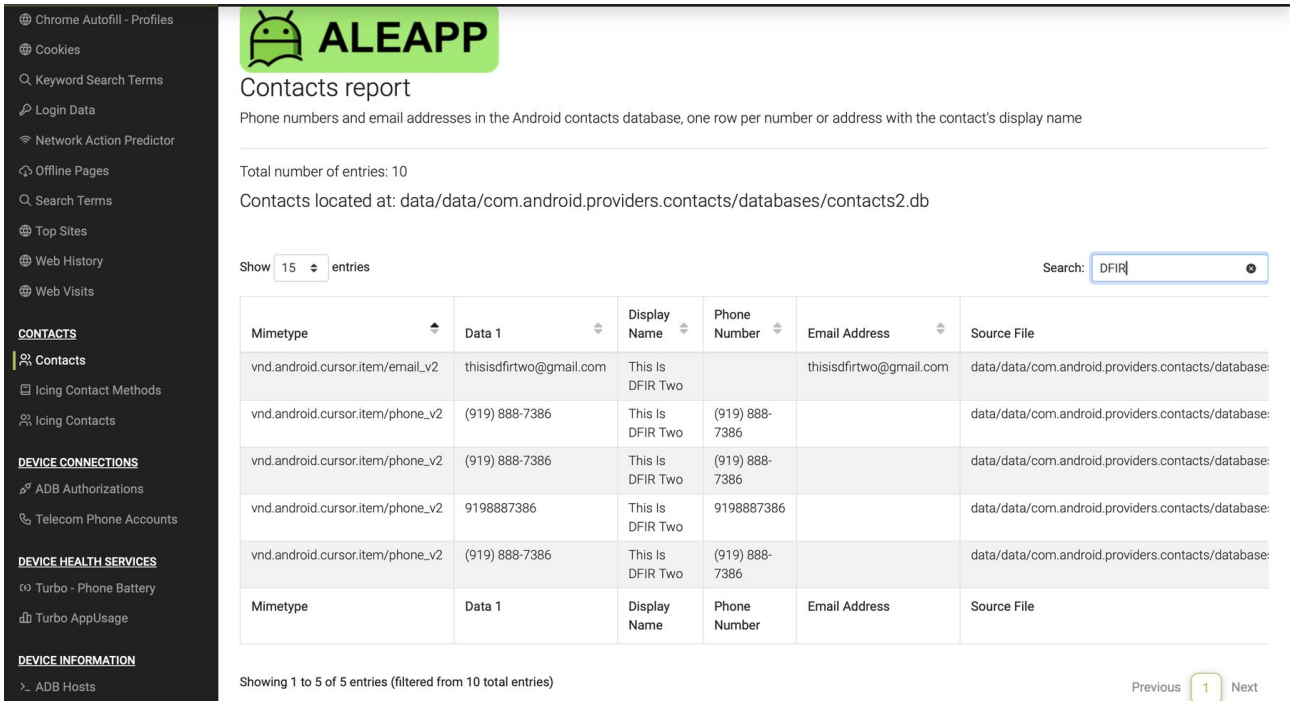


Figure 11. Contacts (filtered on “DFIR”) — This Is DFIR Two, (919) 888-7386 and thisisdfirtwo@gmail.com (source: contacts2.db).

SMS (27 messages) — data/data/com.android.providers.telephony/databases/mmssms.db

Most SMS are **verification codes** (Messenger, Dust, imo, LINE, Signal, Telegram, TikTok, WhatsApp, Snapchat, Zoom). They date the registration of the user to each service, even when the app itself keeps no readable data. The only person-to-person conversation is with This Is DFIR Two on 2020-10-04.

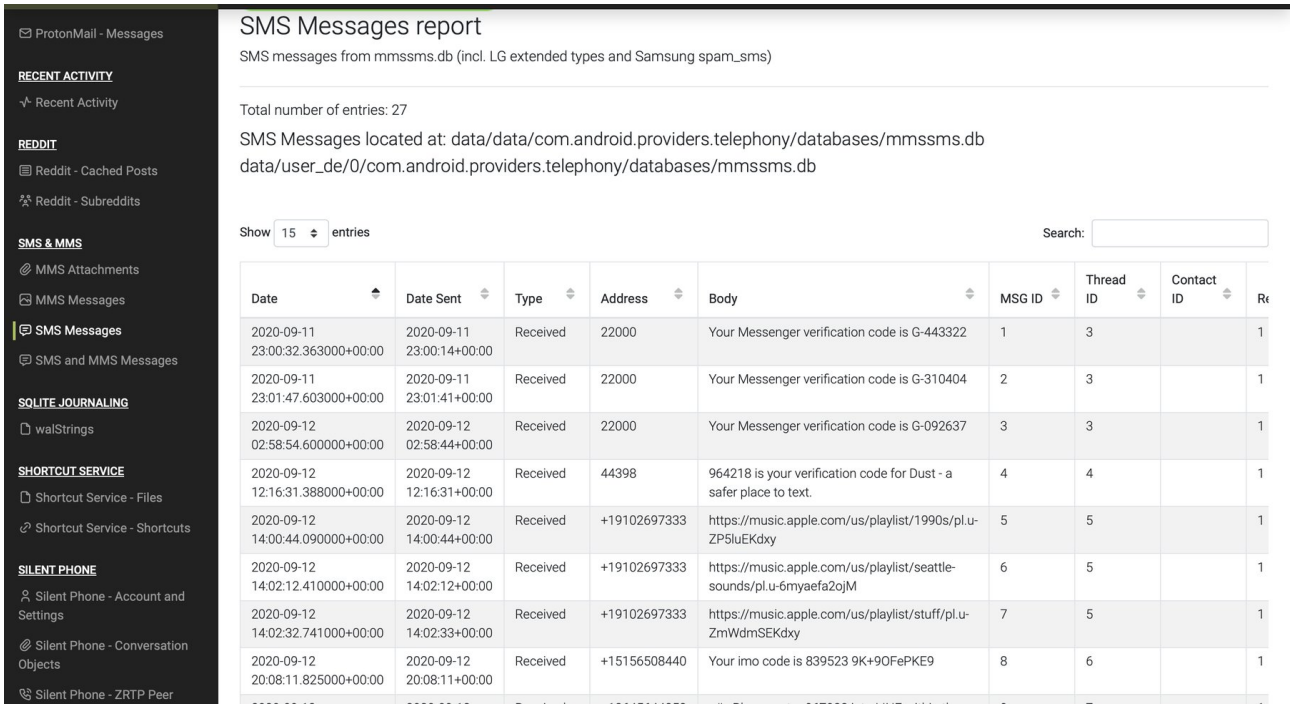


Figure 12. SMS Messages — 27 entries; first SMS 2020-09-11 23:00:32 (Messenger code). ALEAPP lists both mmssms.db paths; section 6.4 shows only the data/data one contains messages.

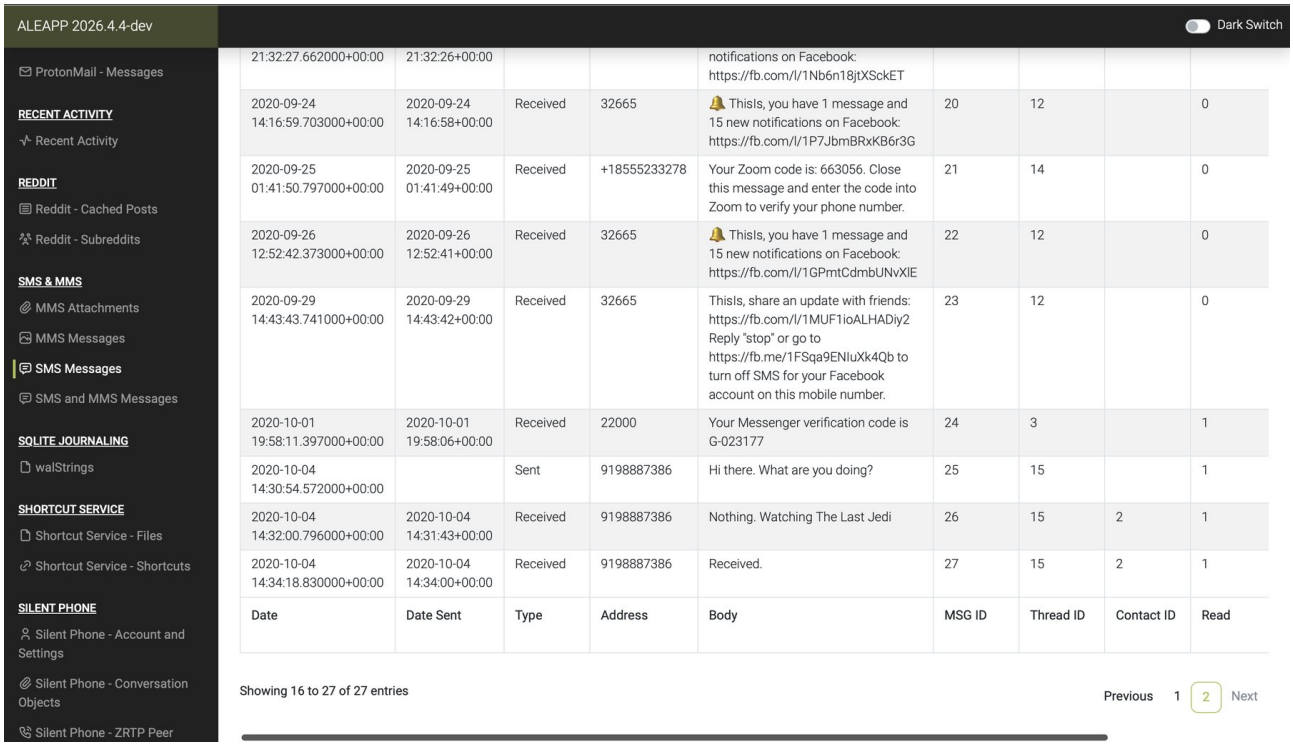


Figure 13. SMS Messages, last rows — conversation with 9198887386 on 2020-10-04 (14:30:54 sent, 14:32:00 and 14:34:18 received).

UTC	Local	Type	Content
14:30:54	10:30:54	SMS sent	“Hi there. What are you doing?”
14:32:00	10:32:00	SMS received	“Nothing. Watching The Last Jedi”
14:33:08	10:33:08	MMS sent	Image (screenshot of a tweet), 62,099 bytes
14:34:18	10:34:18	SMS received	“Received.” — the recipient acknowledges the image
14:37:05	10:37:05	MMS received	Image “Someone had a nope” (snow and a cat), 62,418 bytes

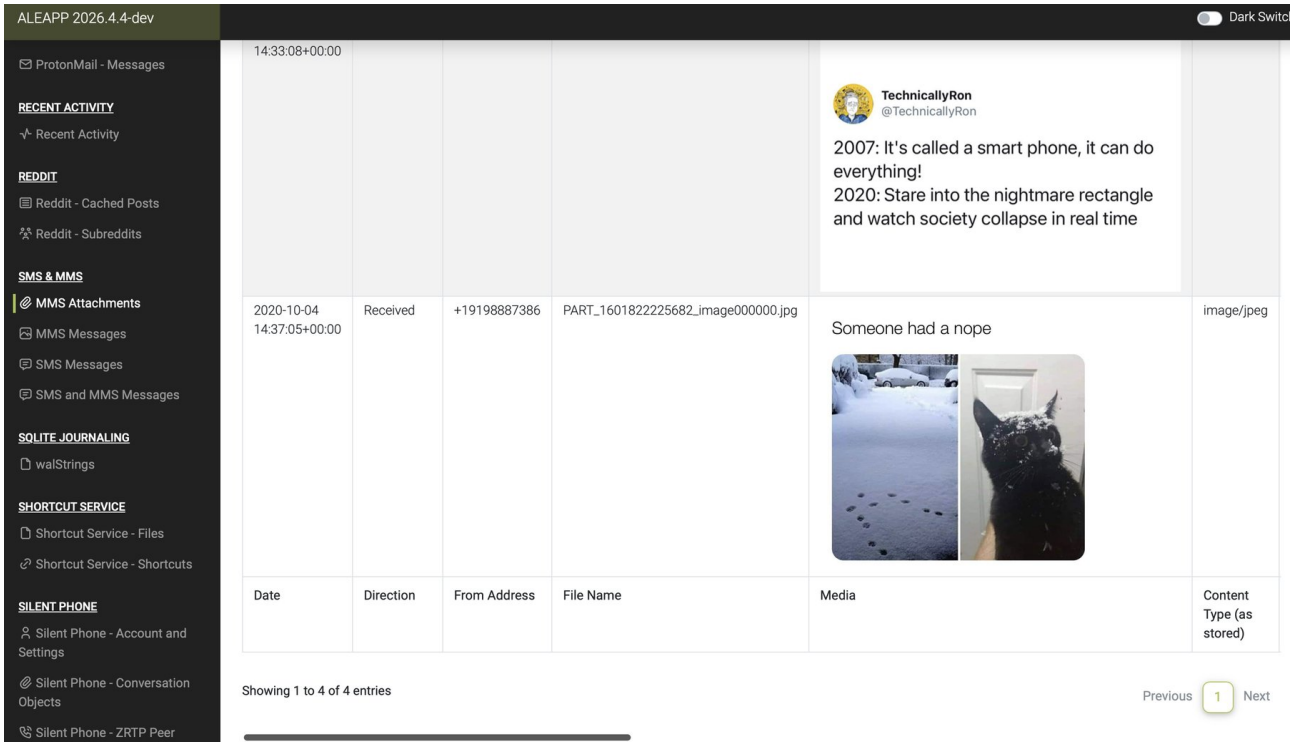


Figure 14. MMS Attachments — the image received on 2020-10-04 14:37:05 from +19198887386 (PART_1601822225682_image000000.jpg) and, above it, the image sent at 14:33:08.

Calls — data/data/com.android.providers.contacts/databases/calllog.db (14 entries)

On 2020-10-04 between 01:51 and 02:37 UTC there are five calls with +1 919-888-7386 (incoming, missed, outgoing; 86–95 s). Google Duo records four calls with This Is DFIR Two at 01:45:47, 01:48:08, 01:51:41 and 01:53:37. The call-log entries of 01:51:41 and 01:53:37 have **no SIM account** and match the Duo times to the second: they are Duo calls written into the system call log, not cellular calls — an example of why sources must be correlated.

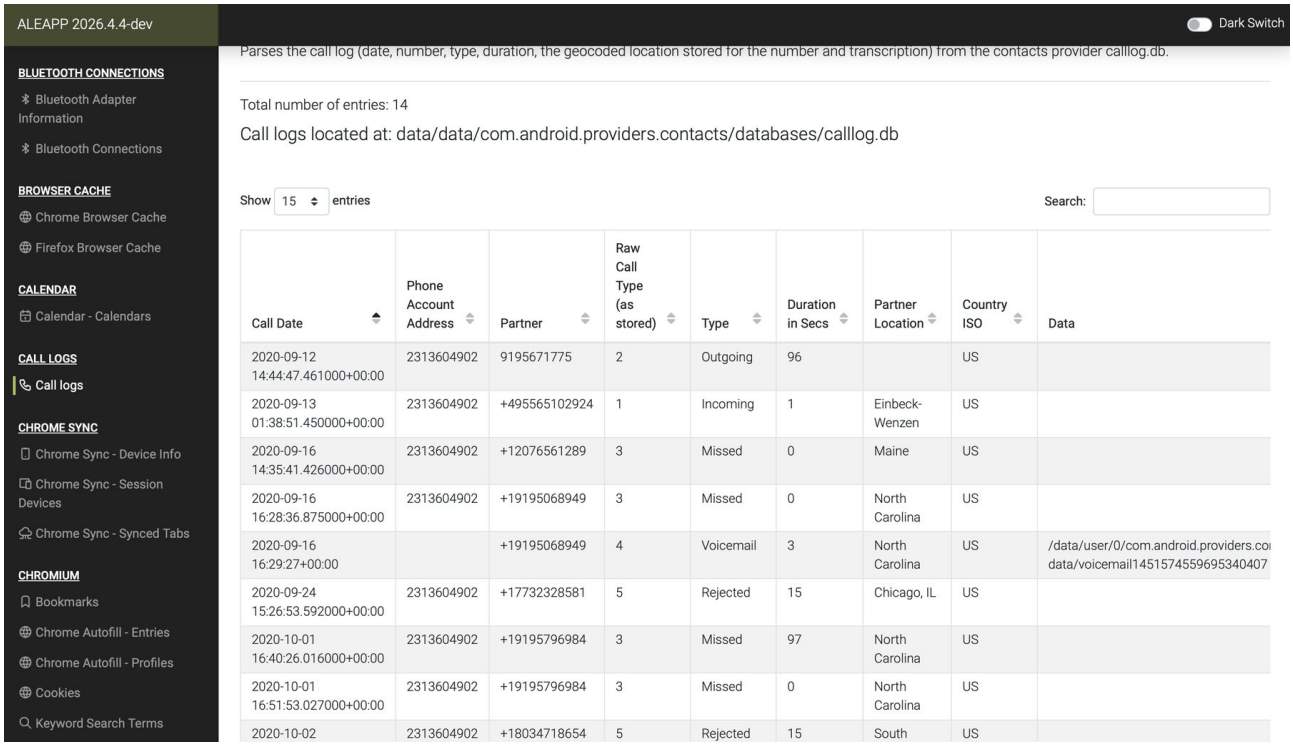


Figure 15. Call logs — first entries of calllog.db (14 entries in total; the calls with This Is DFIR Two are on 2020-10-04).

WhatsApp — data/data/com.whatsapp/databases/msgstore.db

ALEAPP shows 10 messages with 19198887386@s.whatsapp.net: text on 2020-09-23 (“On WhatsApp now.”, “I'm here, too.”), two attachments (encrypted media URLs), and five messages on 2020-10-03 22:42–22:48 with **no text and no attachment**. Section 6.6 shows by manual SQL that these five are **location shares**.

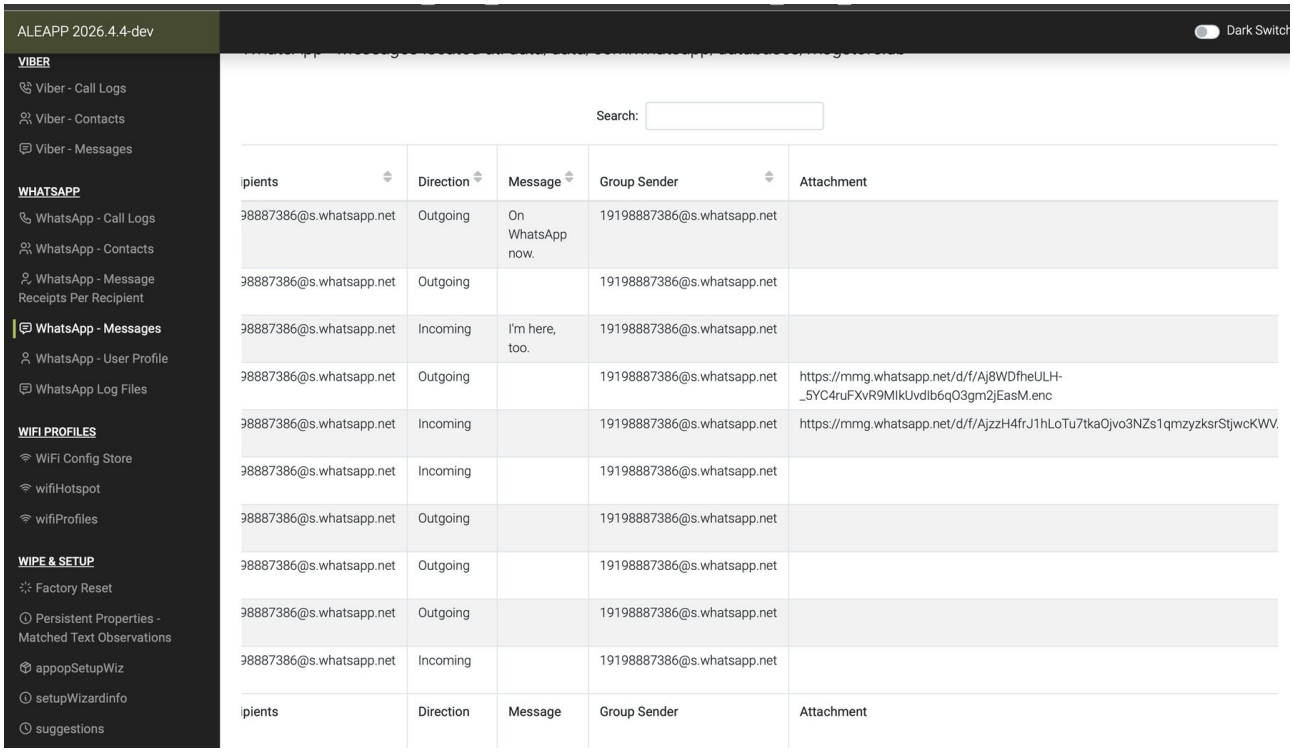


Figure 16. WhatsApp – Messages (ALEAPP): the last five rows (2020-10-03) appear empty in ALEAPP.

Other messaging applications (from the ALEAPP pages of each app)

App	Key content	Date (UTC)
Wire (encrypted)	“What is up with the screen issue?” / “… Let's make this quick.”; image asset from DFIR Two	09-23 13:21–13:46
Kik	“And here's Kik.”; JPEG attachments exchanged both ways	09-21 19:34–19:42
Viber	“Hey there. Off of Silent Phone?” / “Yeah. Finally.” / “Good. Make this one quick.”; media and calls	09-25 01:18–01:29
imo	Photo received (IMG_0123.jpg) and screenshot sent (Screenshot_20200923-110614.png)	09-25 00:24–00:39
Telegram	Chat with DFIR Two, 0 messages stored locally; 4 VoIP call logs	09-22; 10-03
Dust (ephemeral)	Conversation with thisisdfir2 updated 22 minutes before imaging	10-05 19:25
Skype (push notification)	Location shared by DFIR Two (≈ 35.66, −78.83)	10-03 22:57:38
ProtonMail (encrypted e-mail)	Sent “For Android 11” (02:19:14); reply of 1,003,732 bytes with header `X-Attached: D817E4AC-…jpg` (02:21:17)	10-04

5.5 Phase 3 — Files entering and leaving the device (Q3)

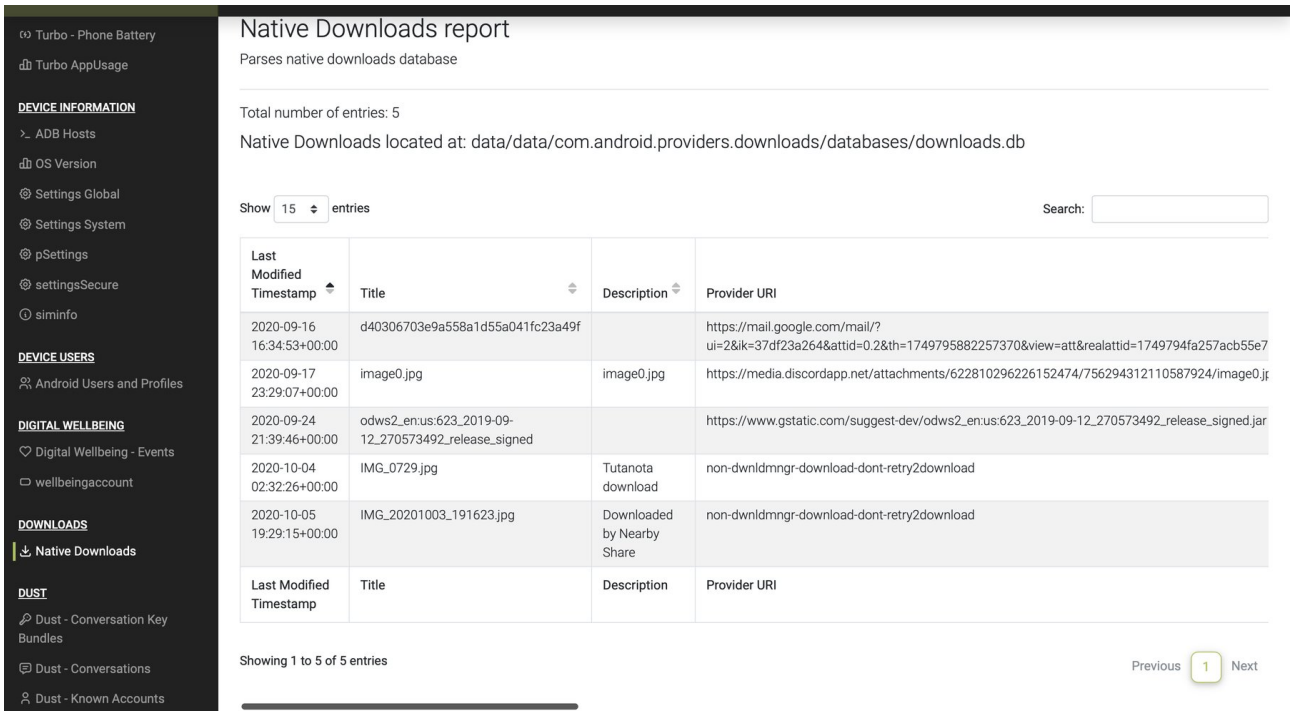


Figure 17. Native Downloads (data/data/com.android.providers.downloads/databases/downloads.db): IMG_0729.jpg “Tutanota download” (2020-10-04 02:32:26) and IMG_20201003_191623.jpg “Downloaded by Nearby Share” (2020-10-05 19:29:15).

Time (UTC)	Channel	Direction	Object	ALEAPP page
2020-09-17 23:29	Discord	In	image0.jpg → Download	Native Downloads
2020-09-17 → 10-03	Imgur app	In	~12 images, e.g. BWV1Yuj.jpg (09-17), uHJlBib.jpg (10-03)	Emulated Storage Metadata
2020-09-21 / 23 / 25	Kik, WhatsApp, Wire, imo, Viber	In / Out	Images and media with DFIR Two	app pages
2020-10-04 01:57–02:06	Google Drive	Cloud	“This is Test Document 12/13/14” created; Document 12 trashed	Cello – Google Drive
2020-10-04 02:19–02:21	ProtonMail	Out / In	“For Android 11” + reply with a JPG attachment	ProtonMail – Messages
2020-10-04 02:32:26	Tutanota	In	IMG_0729.jpg	Native Downloads
2020-10-04 13:05:57	GalleryVault → Google Drive	Out (cloud)	BWV1Yuj.jpg hidden and uploaded	GalleryVault – Cloud Files

Time (UTC)	Channel	Direction	Object	ALEAPP page
2020-10-04 14:33:08	MMS	Out	Image, 62,099 bytes	MMS Attachments
2020-10-05 19:29:15	Nearby Share	In	IMG_20201003_191623.jpg, 3,778,714 bytes	Native Downloads
continuous	Google Photos backup	Out (cloud)	Folders Download, Imgur, Screenshots... configured	Google Photos – Backup Folders

Observation: the Nearby Share photo arrived 3 minutes before the acquisition computer was authorised (19:32:02); the phone was still receiving data just before imaging. **Hypothesis to test:** the MMS image sent on 10-04 (62,099 bytes) has the same size as uHJlBib.jpg downloaded from Imgur on 10-03; comparing their SHA-256 would show whether it is the same file.

Conclusion (Q3): files clearly entered and left the device through many channels, including encrypted ones and cloud storage. Within the scenario, the **channels** are demonstrated; the **confidential nature** of the files is not (they are images and test documents).

5.6 Phase 4 — Concealment (Q4)

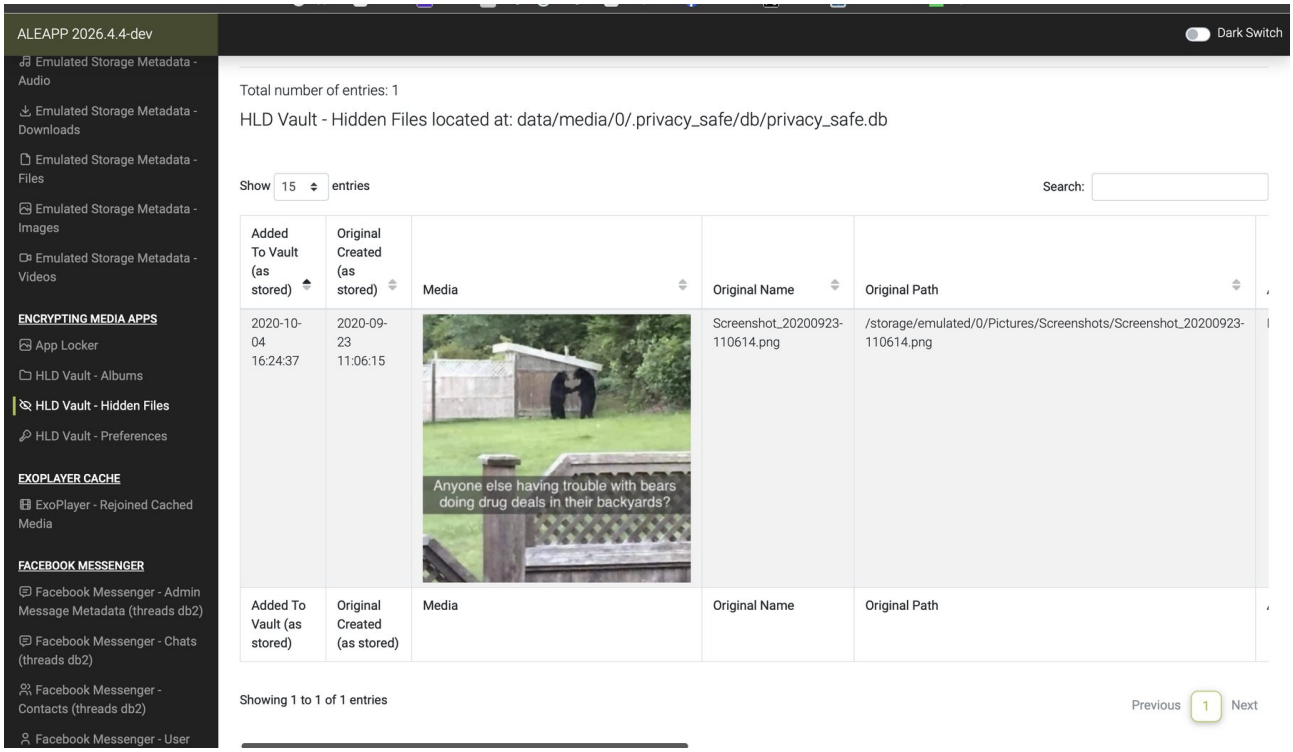


Figure 18. HLD Vault – Hidden Files (data/media/0/.privacy_safe/db/privacy_safe.db): Screenshot_20200923-110614.png, originally in /storage/emulated/0/Pictures/Screenshots/, added to the vault on 2020-10-04 16:24:37 (local time, as stored). ALEAPP recovered the image.

Step	Time	Event	Source
1	2020-09-23 11:06:15 local	Screenshot taken	HLD “Original Created”; file name
2	2020-09-25 00:39:39 UTC	The same screenshot sent through imo	IMO – Messages
3	2020-10-04 16:24:37 local (= 20:24:37 UTC)	Moved into the hidden, encrypted vault folder .privacy_safe	HLD Vault – Hidden Files

GalleryVault (page GalleryVault – Hidden Files (database)): BWV1Yuj.jpg, downloaded from Imgur on 2020-09-17, was added to the vault on 2020-10-04 13:05:57 UTC from /storage/emulated/0/Download/Imgur/, and at the same time uploaded to the vault's Google Drive storage (GalleryVault – Cloud Files). GalleryVault stores its lock PIN partly as an unsalted MD5, `81DC9BDB52D04DC20036DBD8313ED055`, which is the well-known MD5 of “1234”: a hash is not encryption and does not protect a 4-digit secret.

Other indicators: MagiskHide enabled; encrypted or ephemeral messengers (Signal, Wire, Silent Phone, Dust, ProtonMail, Tutanota); Signal and Zoom databases unreadable without keys; a Google Drive document trashed; a guest profile created on 10-04. **Conclusion (Q4):** yes — two vault apps were used on 2020-10-04 and root was hidden; the file-system extraction nevertheless recovered the hidden files, their original names and paths.

5.7 Phase 5 — Location (Q5)

Time (UTC)	Artifact	Location (rounded)	Source
2020-09-12 → 09-14	Maps searches and voice queries (veterinary hospital, Sonic, Homegrown Pizza)	Holly Springs, NC	Google Maps Searches; Quick Search
2020-09-13, 09-20	Android Auto connections (Nissan)	vehicle	Android Auto – Allowed Cars
2020-10-02 → 10-04	Three ~4.9 km runs recorded by a Garmin watch	≈ 35.66, −78.87	Garmin – Polyline Activities
2020-10-03 22:42–22:48	Five WhatsApp location shares (section 6.6)	≈ 35.66, −78.83	msgstore.db (manual SQL)
2020-10-03 22:57:38	Location shared by DFIR Two via Skype	≈ 35.66, −78.83	FCM – Skype and Teams Messages
2020-10-03 23:59 → 10-04 00:02	Last Maps trip to “The Club at 12 Oaks”, Holly Springs	≈ 35.67, −78.88	Google Maps Last Trip

Conclusion (Q5): all sources converge on **Holly Springs, Wake County, North Carolina (USA)**, consistent with the 919 area code and the UTC−4 time zone.

5.8 Phase 6 — Applications and searches (Q6)

The page **Installed Apps → package info** shows about 25 third-party apps installed within ten minutes on **2020-09-12 between 11:50 and 11:59 UTC**, including Imgur, Discord, HLD Vault, Silent Phone, Tor Browser, Wire, Viber, Signal, Telegram, GalleryVault, imo and Kik. Such a burst is consistent with a batch installation. **Google Play Store → Google Play Searches** shows explicit searches for “protonmail” (09-12 20:21) and “tutanota” (09-13 01:21). Browsing on 2020-10-04 concerns news, sport and Star Wars; Chrome stores identity data and one usable password [REDACTED].

6. Manual verification with DB Browser for SQLite (slide 33)

6.1 Why this step

Android apps store their data in **SQLite** databases (single files containing tables). ALEAPP interpreted these databases for us; a forensic examiner must not rely on a single tool. We therefore opened three databases ourselves and checked with SQL that the rows, values and time conversions shown by ALEAPP are really in the files (**cross-validation**). Slide 33 also asks to look for deleted message rows. Databases checked: **mmssms.db** (SMS/MMS), **msgstore.db** (WhatsApp, the example of slide 33) and **accounts_ce.db** (accounts).

6.2 Extraction of the databases from the .tar (Windows, actual commands)

Step 1 — list the files inside the .tar (`tar -tf`) and keep only the lines naming our databases (`Select-String` is the PowerShell equivalent of grep). The result, saved in paths.txt, is the exact list of matching files in the image:

```
PS> tar -tf $TAR | Select-String -Pattern 'mmssms\.db|msgstore\.db|accounts_ce\.db' | ForEach-Object
{ $_.Line } | Set-Content paths.txt
data/data/com.android.providers.telephony/databases/mmssms.db  (+ -journal)
data/data/com.whatsapp/databases/msgstore.db  (+ -wal, -shm)
data/data/com.google.android.gms/databases/icing_mmssms.db, ipa_mmssms.db  (+ -journal)
```

```
data/system_ce/0/accounts_ce.db
data/user/0/...   (same 9 files as data/data/...)
data/user_de/0/com.android.providers.telephony/databases/mmssms.db  (+ -journal)
data/media/0/WhatsApp/Databases/msgstore.db.crypt12
```

Step 2 — extract only those files (`tar -xf $TAR -T paths.txt`), hash them, make them read-only, and copy them to a separate working folder:

```
PS> cd C:\Enquete\SQLite\extracted
PS> tar -xf $TAR -T ..\notes\paths.txt
PS> Get-ChildItem -Recurse -File | Get-FileHash -Algorithm SHA256 | ... | Out-File
..\notes\sqlite_hashes_before.txt
PS> Get-ChildItem -Recurse -File | ForEach-Object { $_.IsReadOnly = $true }
PS> Copy-Item -Recurse .\* ..\working\
```

Extracted file (inside data/)	Size (bytes)	Meaning
data/com.android.providers.telephony/databases/mmssms.db	126,976	Active SMS/MMS database
user_de/0/com.android.providers.telephony/databases/mmssms.db	118,784	Second SMS database (empty, section 6.4)
mmssms.db-journal, icing/ipa_mmssms.db-journal (7 files)	0	Empty rollback journals: no transaction in progress
data/com.whatsapp/databases/msgstore.db	851,968	WhatsApp messages
msgstore.db-wal	107,152	Write-Ahead Log: recent changes not yet merged into msgstore.db
msgstore.db-shm	32,768	Shared-memory index of the WAL
media/0/WhatsApp/Databases/msgstore.db.crypt12	40,338	Encrypted WhatsApp backup (decrypted by the agent, section 7.2)
data/com.google.android.gms/databases/icing_mmssms.db, ipa_mmssms.db	53,248 / 28,672	Google Play Services search indexes of SMS (secondary source)
system_ce/0/accounts_ce.db	385,024	Accounts

6.3 Two paths, same files: data/data and data/user/0

Grouping the extracted files by SHA-256 shows that each file in data/data has an identical twin in data/user/0 (for example msgstore.db: FF469756…6F4E in both). On Android, `/data/user/0` is a symbolic link to `/data/data`; the acquisition tool saved the folder under both names. Only data/data is analysed. The 7 journal files share E3B0C442…B855, the SHA-256 of an **empty file**.

```
PS> Get-ChildItem -Recurse -File | Get-FileHash -Algorithm SHA256 | Group-Object Hash | Where-Object
Count -gt 1 | ...
DA5D0D16…52A7F9  data\data\...\mmssms.db            = data\user\0\...\mmssms.db
E3B0C442…7852B855  7 x *-journal files (empty file hash)
A9B77ADB…19E6FD3A  icing_mmssms.db (data\data = data\user\0)
035AFFC9…3775DD8  ipa_mmssms.db   (data\data = data\user\0)
FF469756…96F4E    msgstore.db     (data\data = data\user\0)
18801C9B…96F65F6  msgstore.db-shm (data\data = data\user\0)
69D879F5…374EB76  msgstore.db-wal (data\data = data\user\0)
```

6.4 The two SMS databases

Each database was opened with **File → Open Database Read Only** (status bar “Lecture seule”). The first one, in `user_de`, has the full SMS/MMS schema (18 tables: sms, pdu, part, addr, threads, words…).

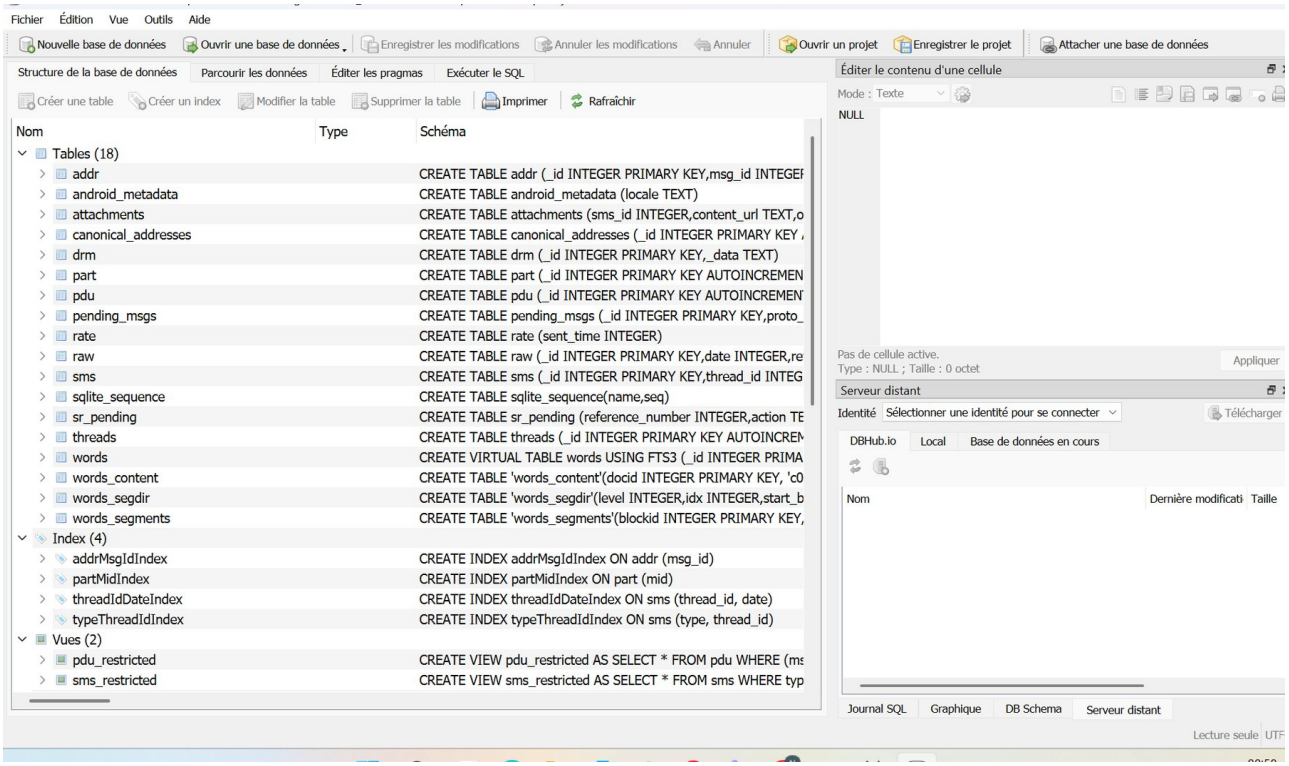


Figure 19. DB Browser — structure of data/user_de/0/.../mmssms.db, opened read-only: 18 tables, 4 indexes, 2 views.

The following query counts rows and reads the database's own page counters (`page_count` = total 4,096-byte pages; `freelist_count` = pages released by deletions, which may still hold old data):

```
SELECT (SELECT COUNT(*) FROM sms) AS nb_sms, (SELECT COUNT(*) FROM pdu) AS nb_mms,
       (SELECT COUNT(*) FROM threads) AS nb_threads,
       (SELECT page_count FROM pragma_page_count) AS total_pages,
       (SELECT freelist_count FROM pragma_freelist_count) AS free_pages;
```

Database	SMS	MMS	Threads	Pages (4,096 B)	Free pages	Conclusion
user_de/0/.../mmssms.db	0	0	0	29 (118,784 B)	0	Schema only, never used
data/data/.../mmssms.db	**27**	**2**	15	31 (126,976 B)	0	**Active database**

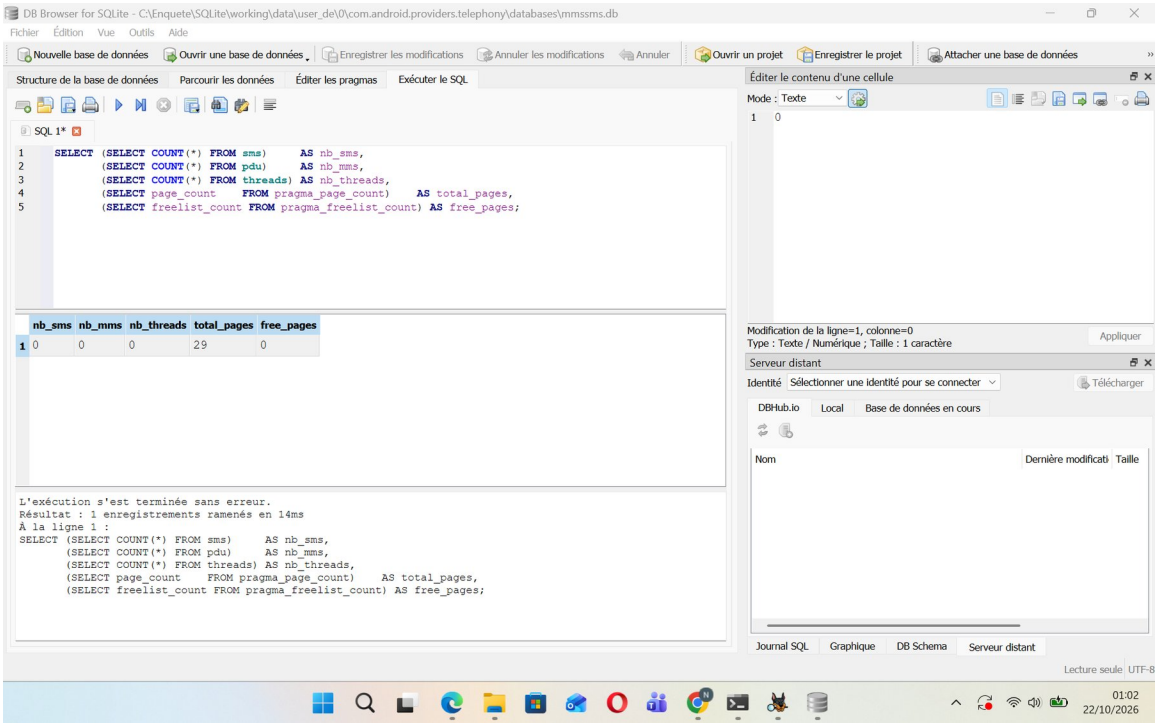


Figure 20. user_de mmssms.db: 0 SMS, 0 MMS, 0 threads, 29 pages, 0 free pages.

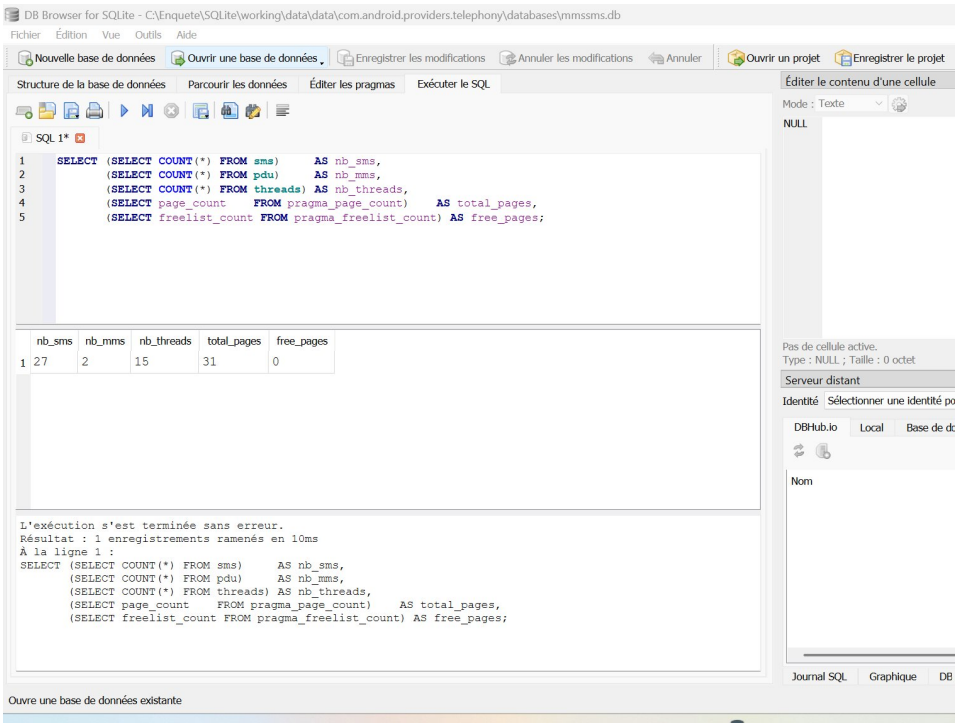


Figure 21. data/data mmssms.db: 27 SMS, 2 MMS, 15 threads, 31 pages, 0 free pages — same counts as ALEAPP.

All SMS were then listed with readable dates. `date` is stored in **milliseconds** since 1970-01-01 (Unix time): dividing by 1000 gives seconds and `datetime(…,'unixepoch')` converts to UTC; `'-4 hours'` gives local time; type 1 = received, 2 = sent. Worked example: 1599865232363 ms → 1,599,865,232 s → **2020-09-11 23:00:32 UTC** (19:00:32 local).

```
SELECT _id, address,
       datetime(date/1000,'unixepoch')            AS date_utc,
       datetime(date/1000,'unixepoch','-4 hours') AS date_local,
       CASE type WHEN 1 THEN 'Received' WHEN 2 THEN 'Sent' ELSE type END AS direction,
       body, date AS raw_date
FROM sms ORDER BY date;
```

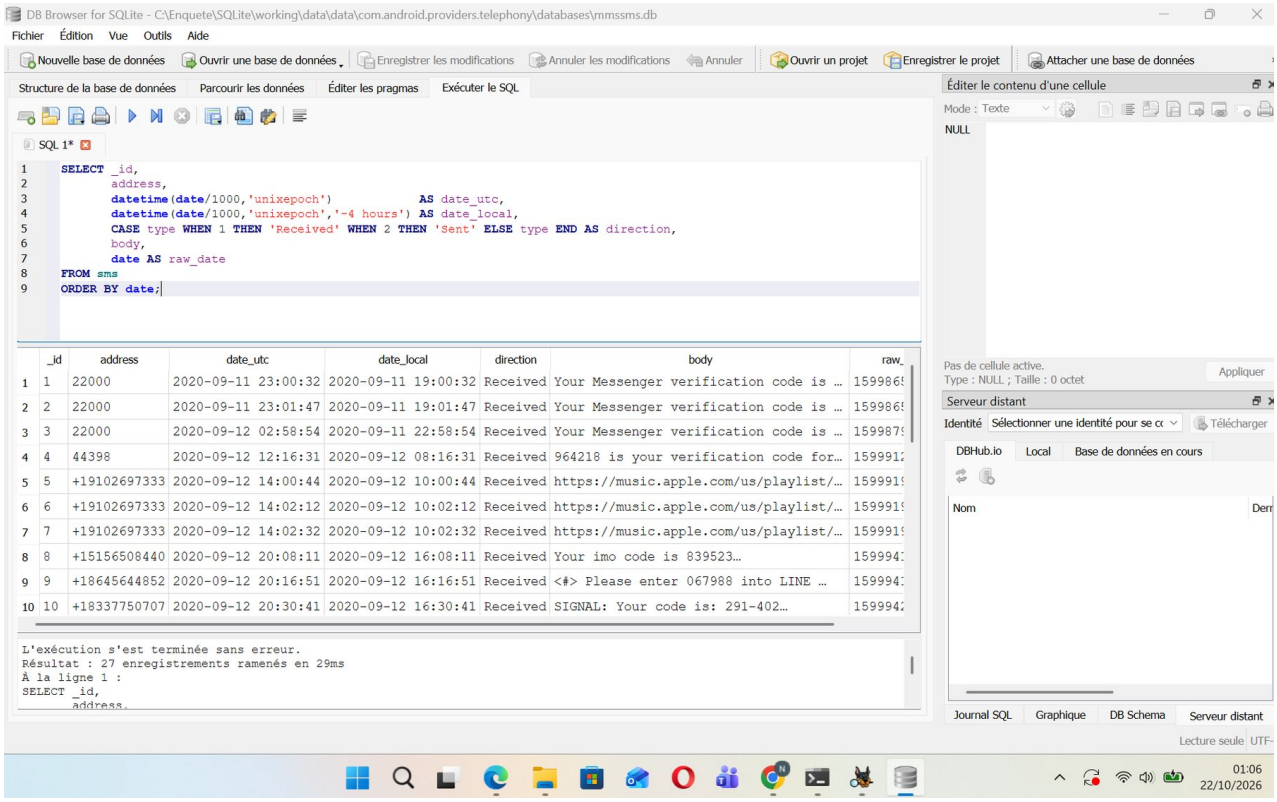


Figure 22. Manual SQL: first SMS rows — identical to ALEAPP (row 1: 22000, 2020-09-11 23:00:32 UTC, Messenger code). Result: 27 rows.

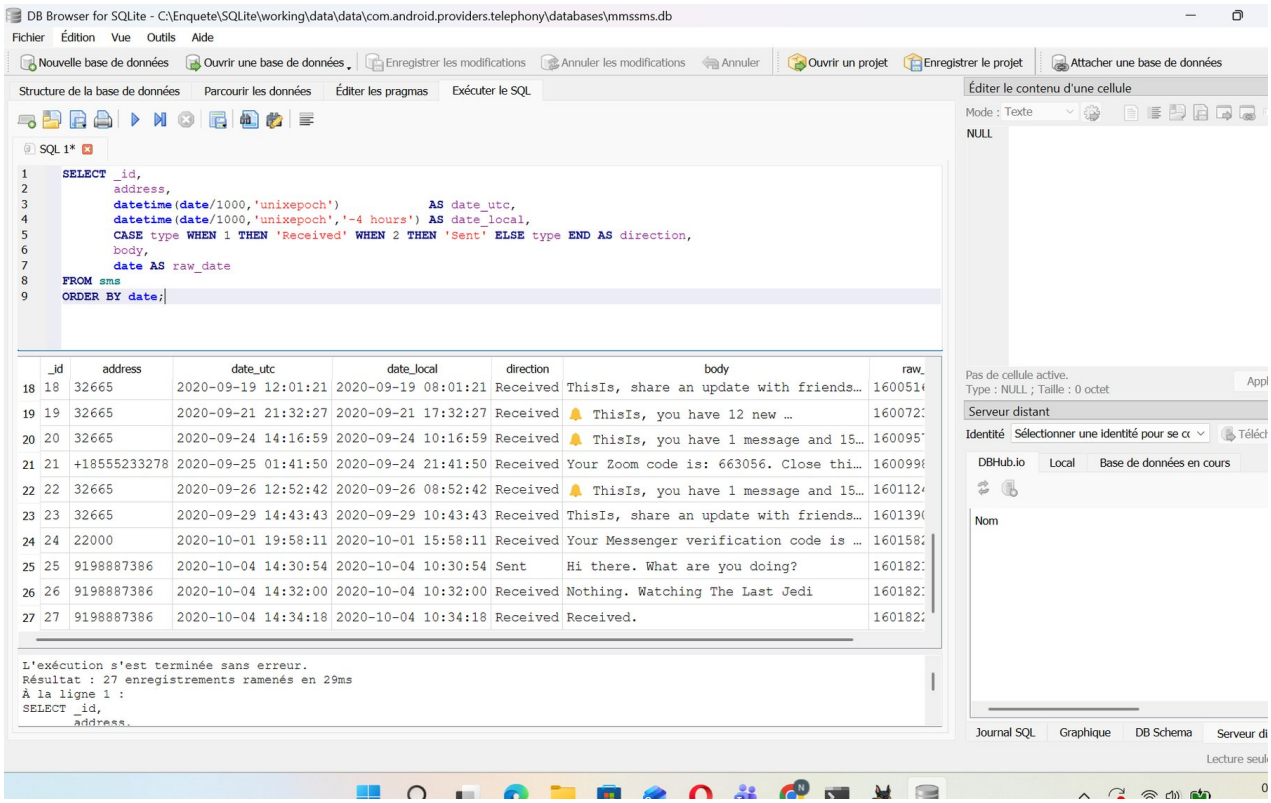


Figure 23. Manual SQL: last rows — the 2020-10-04 conversation with 9198887386, identical to ALEAPP.

6.5 MMS and a trace of deletion

An MMS is stored in two tables: **pdu** (one row per MMS; `date` in **seconds**, not milliseconds; `msg_box` 1 = received, 2 = sent) and **part** (the pieces of each MMS; `mid` = the MMS number). They were joined:

```
SELECT p._id AS mms_id, datetime(p.date,'unixepoch') AS date_utc, ...,
       pt._id AS part_id, pt.ct AS content_type, pt.cl AS file_name, pt.text, pt._data AS file_path
FROM pdu p JOIN part pt ON pt.mid = p._id ORDER BY p.date, pt.seq;
```

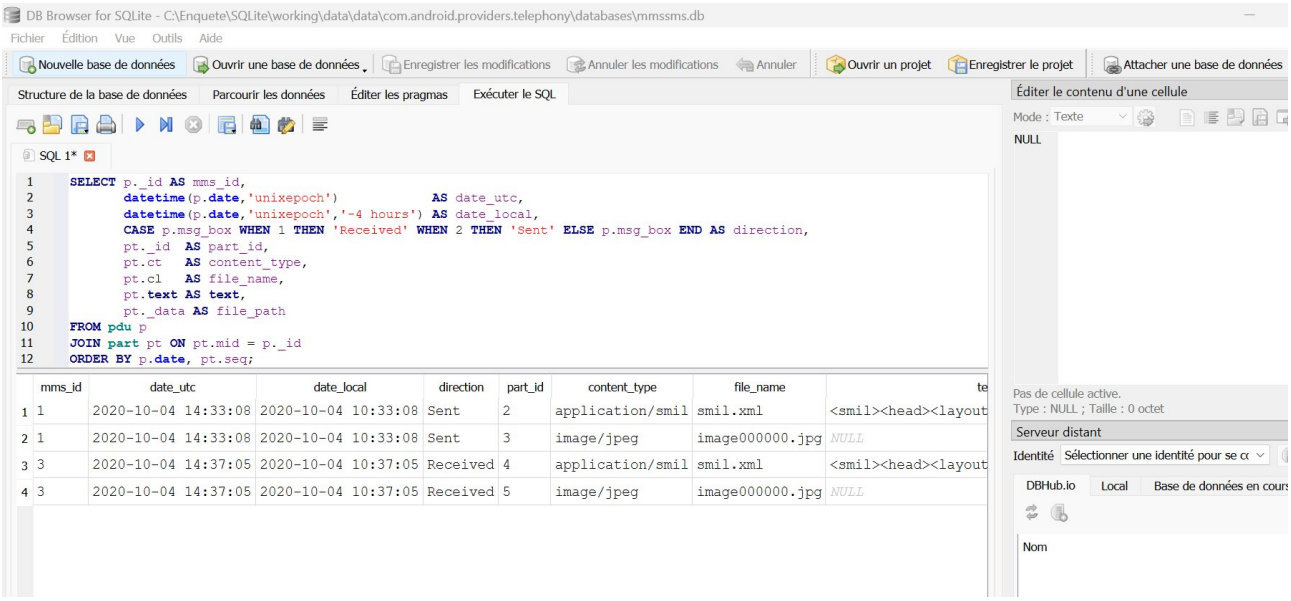


Figure 24. Manual SQL: MMS 1 (sent 2020-10-04 14:33:08) and MMS 3 (received 14:37:05), each with a SMIL layout part and an image/jpeg part.

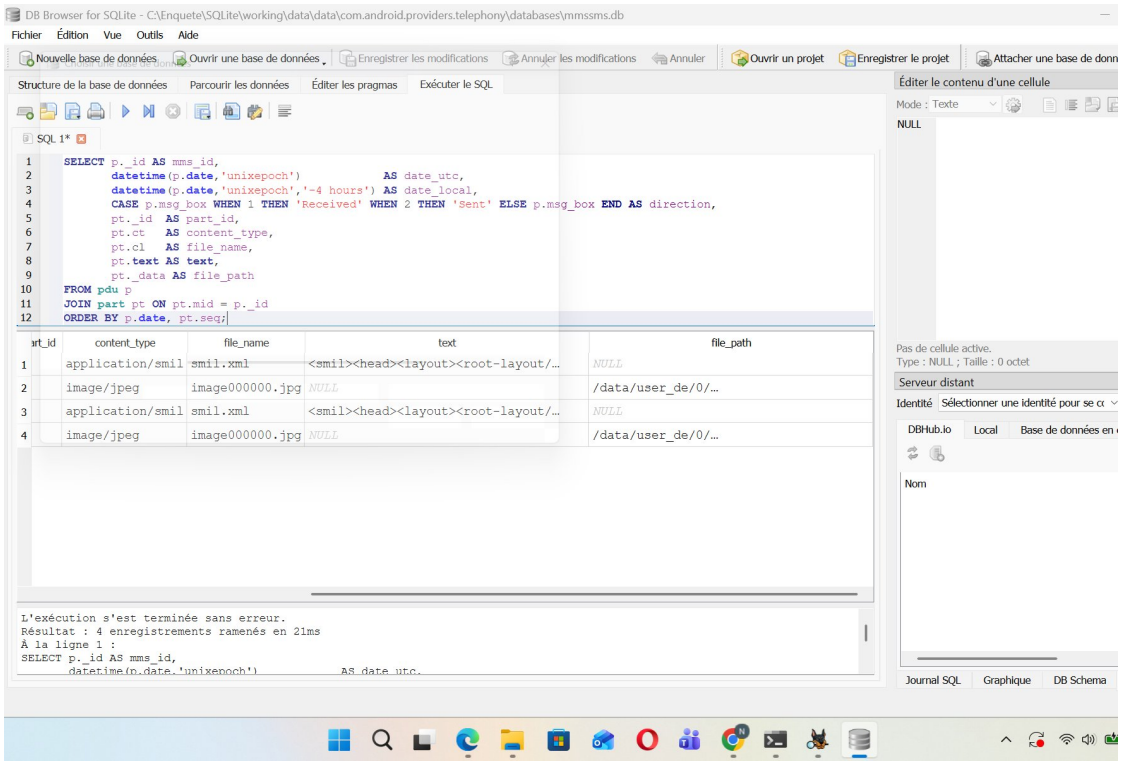


Figure 25. Same query, right-hand columns: the image parts point to files under /data/user_de/0/... (the app_parts folder).

The MMS numbers are **1 and 3** (no 2). The join shows the parts 2–5; part 1 is not shown because it is not linked to an existing MMS (section 6.5.1). These tables use AUTOINCREMENT: SQLite never reuses a number and records the highest number ever given in the internal table `sqlite_sequence`:

```
SELECT name, seq FROM sqlite_sequence;
```

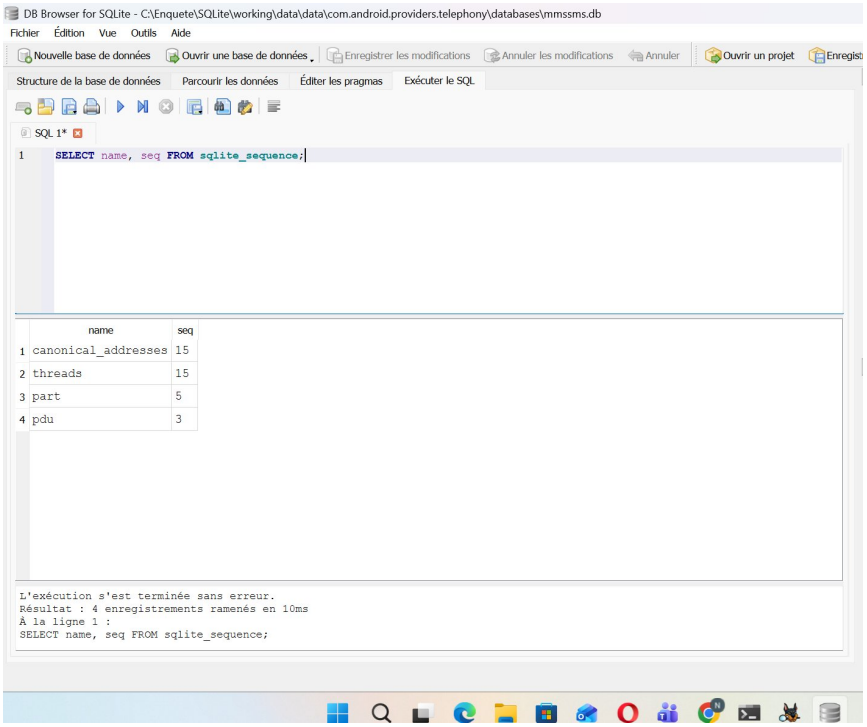


Figure 26. sqlite_sequence: pdu = 3 while only 2 MMS exist (MMS 2 deleted); part = 5 and the part table has 5 rows (no part deleted); threads = 15 = 15 threads.

> **Finding: a deleted MMS**
> MMS identifier 2 was allocated but no longer exists: an MMS was created and later removed (deleted message, cancelled draft or failed send). Because the database has 0 free pages and an empty journal, its content is not recoverable from mmssms.db. This is a **trace of deletion** (slide 33) even though the content is gone.

6.5.1 Correction and an orphan MMS part (found by the verification agent, section 7.1)

A first reading of the join suggested that part 1 had also been deleted. The verification agent showed that this is **wrong**: the part table still has 5 rows. Part 1 is an **orphan part**: its parent identifier is `mid = 2047483647`, a placeholder value that is not a real MMS number, and it points to the file `app_parts/PART_1599870740204`. The number in the file name is a Unix time in milliseconds: **2020-09-12 00:32:20 UTC** (11 September, 20:32 local). Interpretation (hypothesis): an attachment prepared in the messaging app but never linked to a stored message. ALEAPP also lists this file among the MMS attachments without a message. It is not the deleted MMS 2, which dates from a different context.

6.6 WhatsApp: the “empty” messages are location shares

msgstore.db was opened read-only; DB Browser automatically applied the `-wal` file, so the most recent changes are visible. The database has 111 tables and contains **both** the old WhatsApp schema (`messages`) and the new one (`message` + specialised tables), because WhatsApp was migrating in 2020.

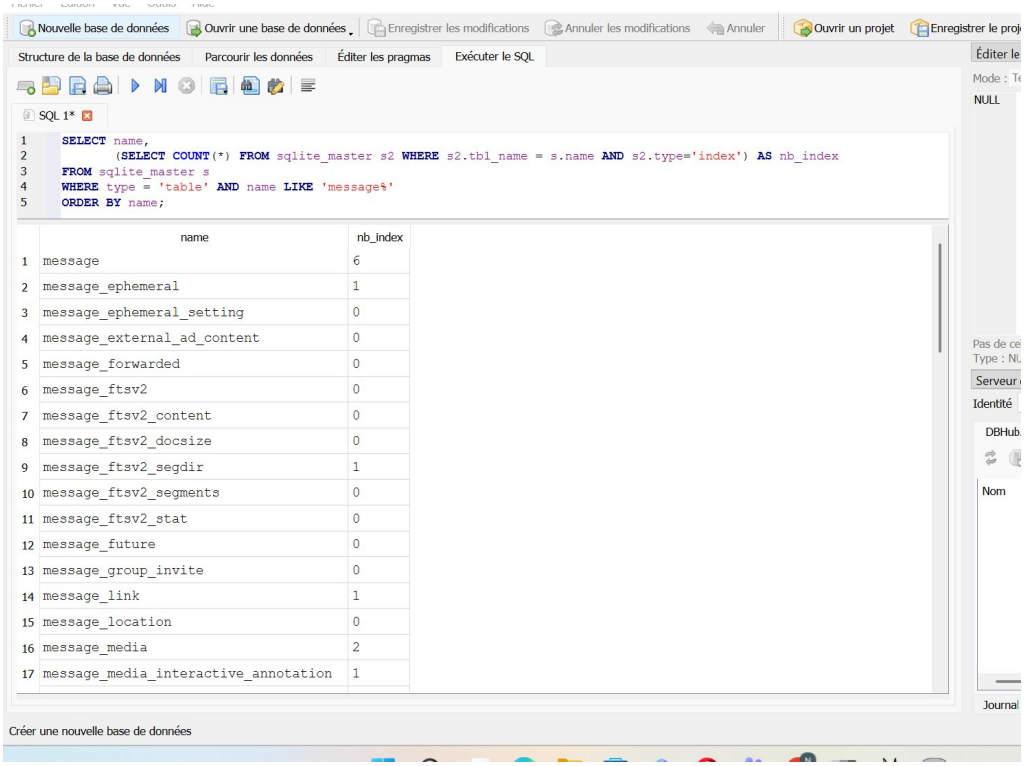


Figure 27. sqlite_master query: tables whose name starts with “message” — both message (new schema) and messages (old schema) exist.

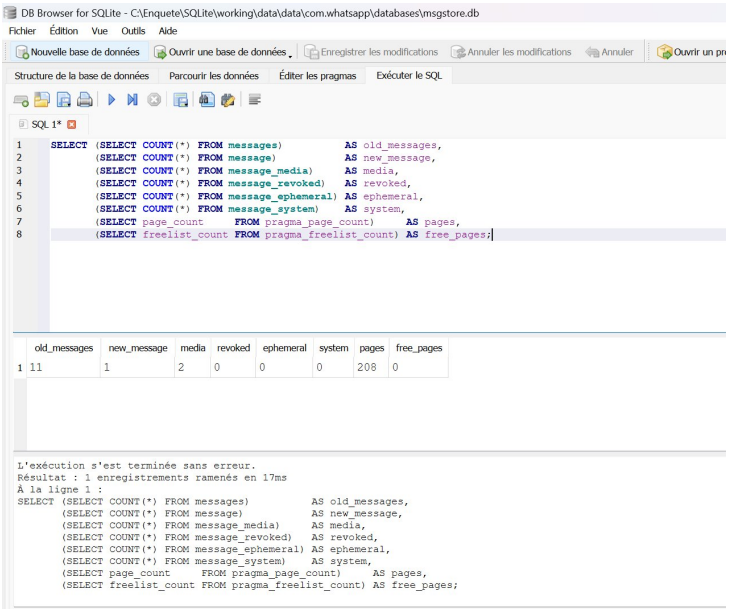


Figure 28. Row counts: messages = 11, message = 1, media = 2, revoked = 0, ephemeral = 0, system = 0; 208 pages, 0 free pages.

11 rows = the 10 messages shown by ALEAPP + 1 internal WhatsApp placeholder row (_id 1, date 1970). **revoked = 0** means no message was deleted “for everyone”. Listing all messages with the type code `media_wa_type`:

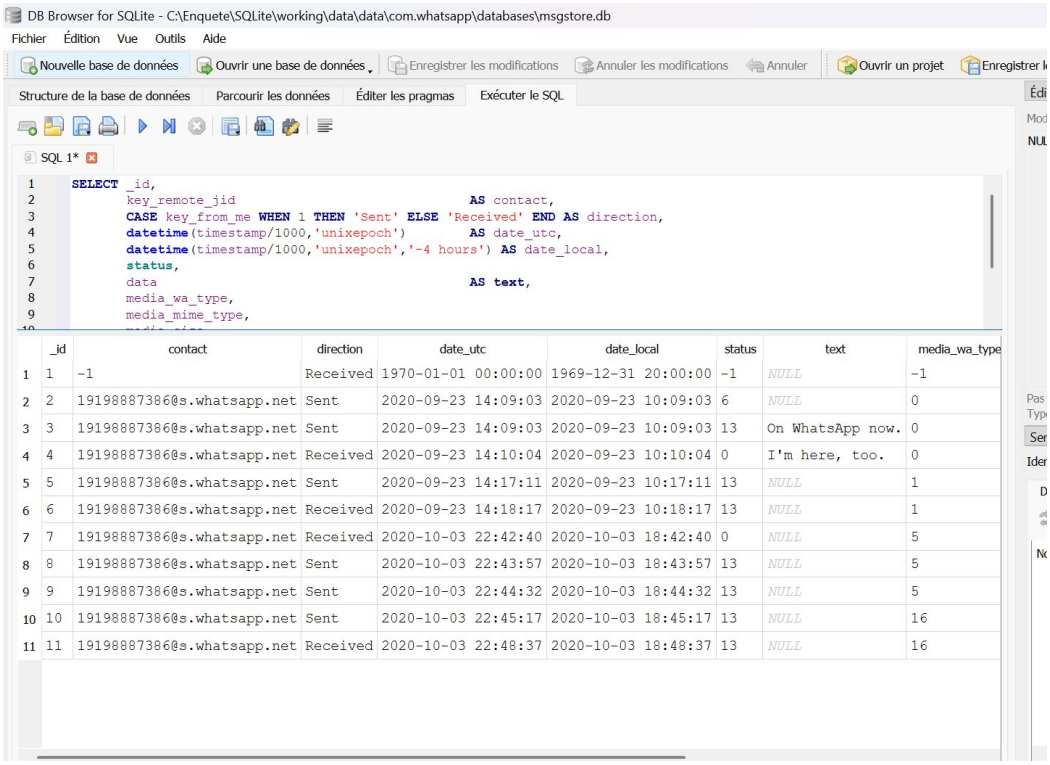


Figure 29. All rows of messages: the 2020-10-03 rows have media_wa_type 5 and 16.

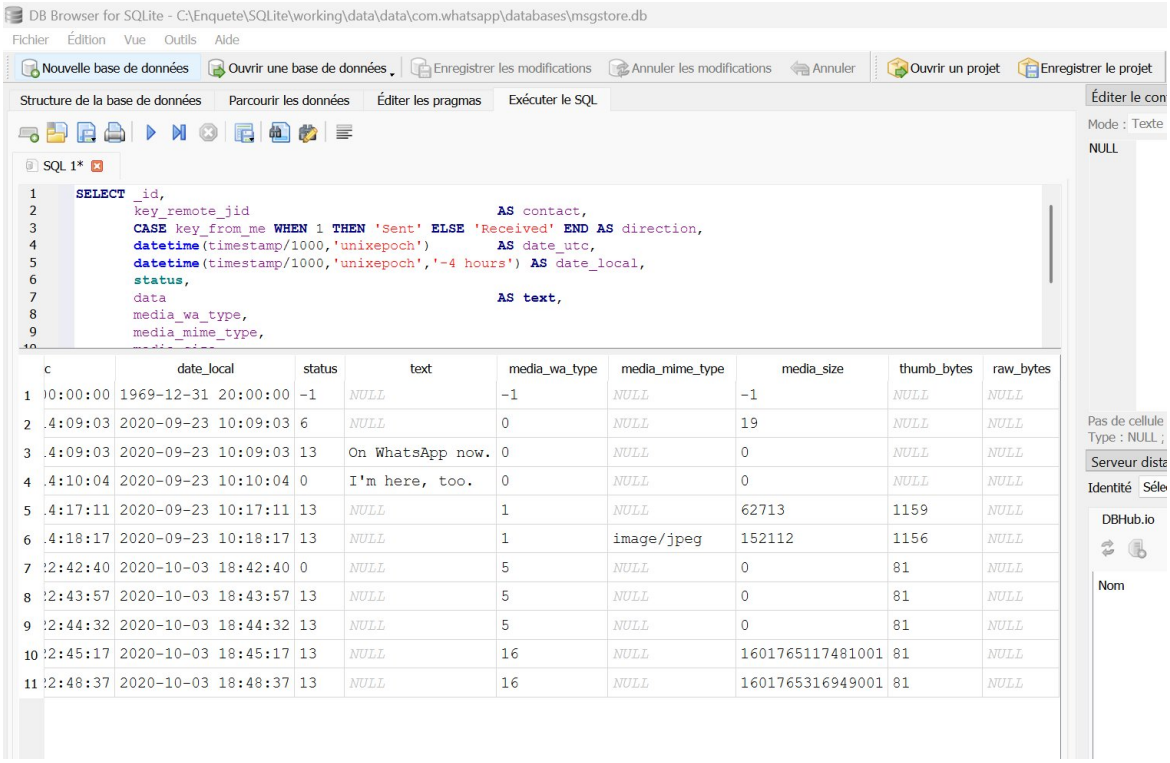


Figure 30. Right-hand columns: rows 10–11 hold an identifier in media_size (1601765117481… = 2020-10-03 22:45:17.481 UTC, the message time), 81-byte binary data on each location row.

_id	UTC	Direction	media_wa_type	Meaning
1	1970-01-01	—	-1	WhatsApp internal placeholder row (not a message)
2	2020-09-23 14:09:03	Sent	0, status 6	System message (probably the end-to-end encryption notice)
3–4	2020-09-23 14:09–14:10	Sent / Received	0 = text	“On WhatsApp now.” / “I'm here, too.”

_id	UTC	Direction	media_wa_type	Meaning
5–6	2020-09-23 14:17–14:18	Sent / Received	1 = image	62,713 and 152,112 bytes
7–9	2020-10-03 22:42–22:44	Received, Sent, Sent	**5 = location**	Static location shares
10–11	2020-10-03 22:45–22:48	Sent, Received	**16 = live location**	Live location shares

```
SELECT _id, ..., CASE media_wa_type WHEN 5 THEN 'Location' WHEN 16 THEN 'Live location' END AS type,
       latitude, longitude, media_duration, media_name, media_caption
FROM messages WHERE media_wa_type IN (5, 16) ORDER BY timestamp;
```

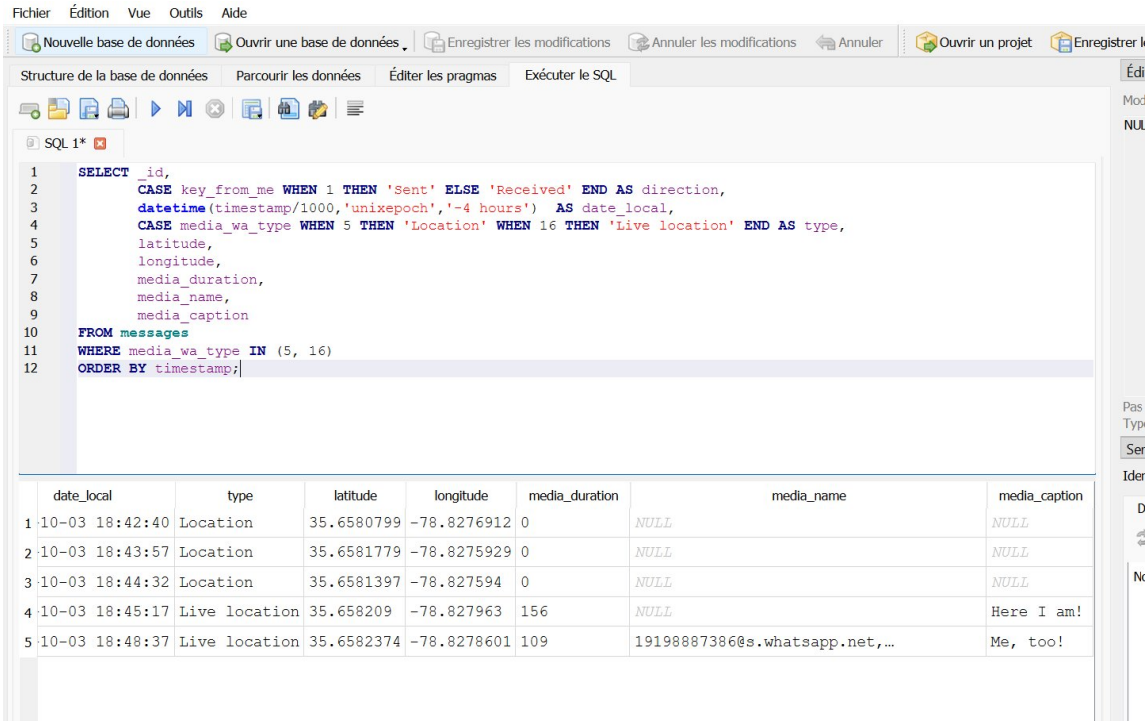


Figure 31. Location messages: coordinates and captions “Here I am!” (sent, 18:45:17 local) and “Me, too!” (received, 18:48:37 local). Precise coordinates are rounded in the text.

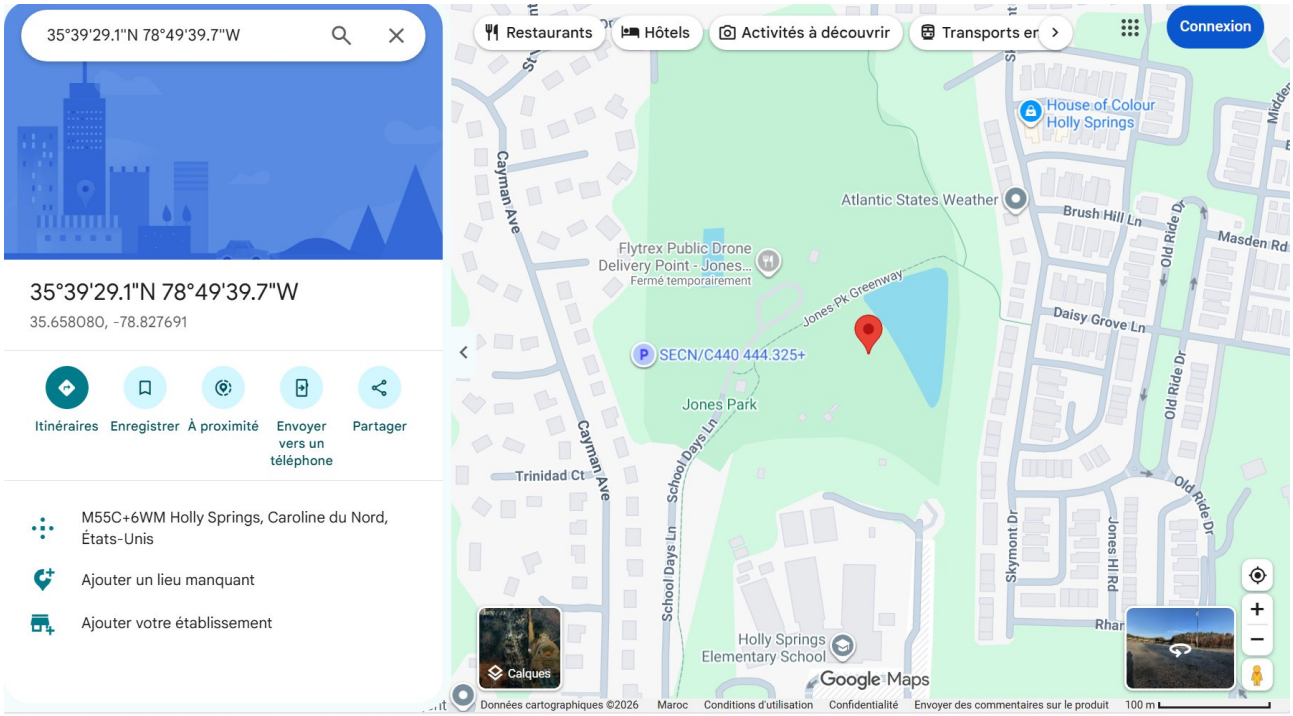


Figure 32. Google Maps, coordinates 35.658080, −78.827691 (first WhatsApp location, rounded): Jones Park, Holly Springs, North Carolina. The coordinates come from msgstore.db; the town name comes from placing them on a map and is confirmed by the Maps last trip and searches.

> **Finding missed by the automated tool**
>
> The five WhatsApp messages that ALEAPP showed as empty are three location shares and two live-location shares exchanged with This Is DFIR Two on 2020-10-03 between 18:42 and 18:48 local time. All five positions lie within about 35 m of each other (largest distance between two points: 34 m), at ≈ 35.66 N, −78.83 W (Holly Springs, NC); the captions “Here I am!” / “Me, too!” show a meeting being coordinated. Ten minutes later a location was also shared through Skype. **Only the manual SQL analysis revealed this.**

6.7 Accounts

```
SELECT _id, name, type FROM accounts ORDER BY _id;      -- password column deliberately not selected
```

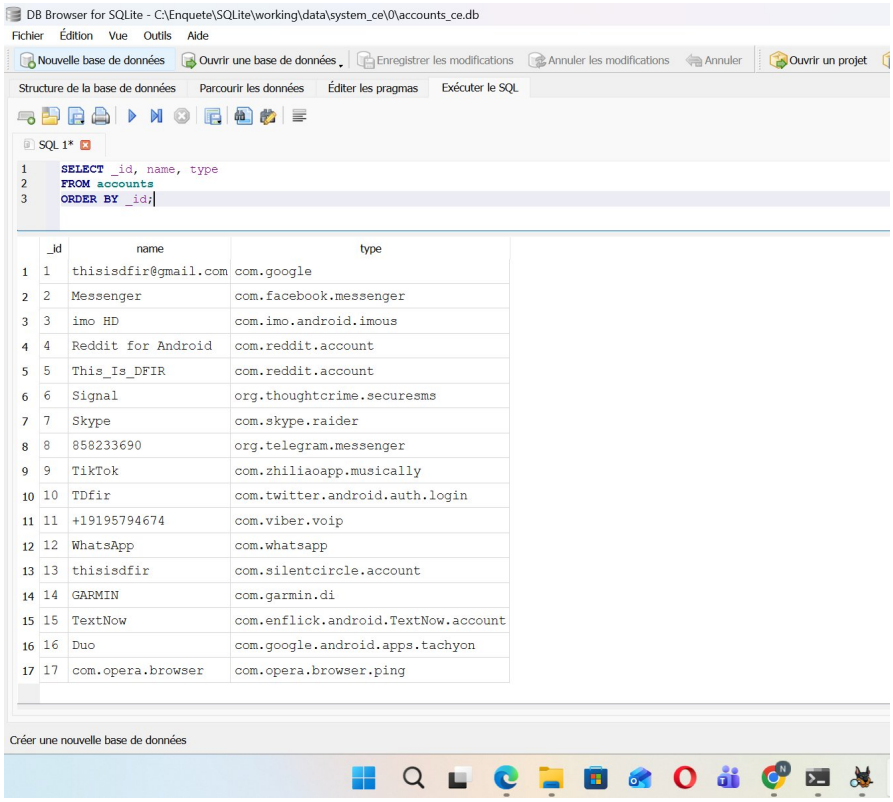


Figure 33. accounts_ce.db: 17 accounts, identical to ALEAPP (thisisdfir@gmail.com, Messenger, imo, Reddit ×2, Signal, Skype, Telegram, TikTok, Twitter, Viber, WhatsApp, Silent Phone, Garmin, TextNow, Duo, Opera).

The schema also contains a **trigger** `accountsDelete`: when an account is deleted, SQLite automatically deletes its tokens (authtokens) and extras — removing an account also removes its traces in the related tables.

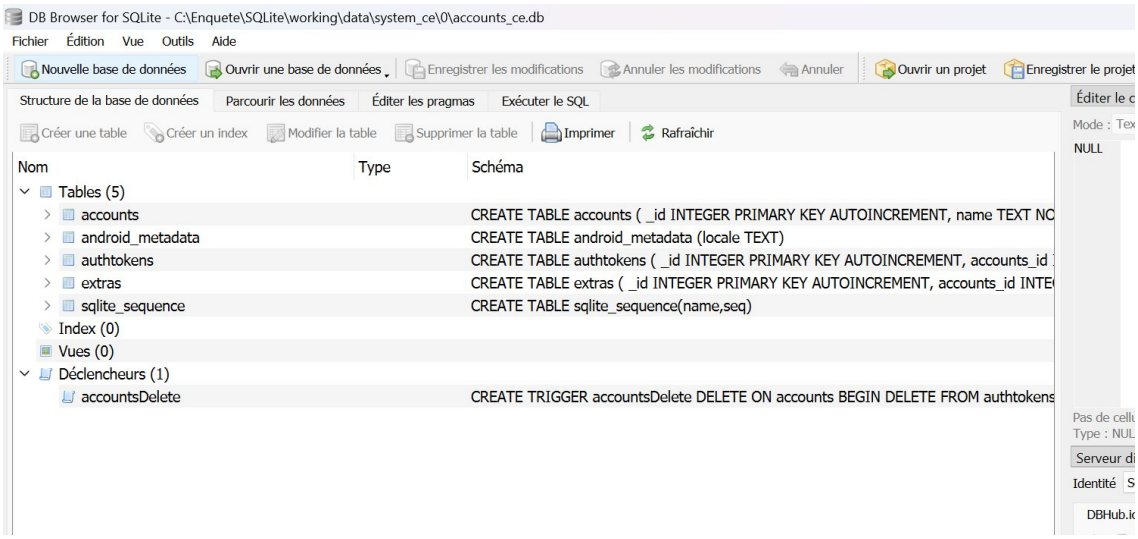


Figure 34. accounts_ce.db structure: 5 tables and the trigger accountsDelete.

6.8 Integrity of the manual work

```
PS> ... Get-FileHash ... | Out-File ..\notes\sqlite_hashes_after.txt
PS> if ((Get-Content before -Raw) -eq (Get-Content after -Raw)) { 'IDENTICAL' } else { 'DIFFERENT' }
IDENTICAL
PS> Compare-Object $extracted.Hash $working.Hash
77FFC33CBB8D5181F68653AFA74D8A31A2E40C759BC3B154610309513FFC5FAF  =>   (working copy)
18801C9B19CB58A496ADFC0BECCB6F99D498C81C2AA86A6FFF0F9046096F65F6  <=   (extracted copy)
```

The 22 extracted files are **identical** before and after the analysis. Only one working file changed: `msgstore.db-shm` (18801C9B… → 77FFC33C…). The -shm file is the shared-memory index of the

WAL, which SQLite rebuilds when it opens a WAL database, **even in read-only mode**. This shows concretely why the analysis is done on copies.



Figure 35. PowerShell: hashes of the extracted databases before and after the analysis — IDENTICAL.

7. Cross-validation summary: ALEAPP versus manual verification

Check	ALEAPP	Manual (DB Browser / SQL)	Match
Number of SMS	27	27	Yes
First SMS (UTC)	2020-09-11 23:00:32.363	2020-09-11 23:00:32 (raw 1599865232363 ms)	Yes
Conversation of 2020-10-04 (3 SMS)	Present	Present, identical text and times	Yes
Number of MMS / image parts	2 / 2	2 / 2 (+ 2 SMIL layout parts)	Yes
Deleted MMS	Not reported	sqlite_sequence: MMS 2 deleted; part 1 is an orphan part (not deleted)	New finding
Which mmssms.db is active	Both paths listed	user_de empty; data/data active	Clarified
WhatsApp messages	10	11 rows = 10 + 1 internal row	Yes
WhatsApp 2020-10-03 (5 messages)	Shown empty	Location / live location, coordinates, captions	New finding
Number of accounts	17	17	Yes
Free pages (deleted data in freelist)	—	0 in both mmssms.db and msgstore.db	—
Integrity of extracted databases	—	IDENTICAL before/after	—

Conclusion: every value checked is identical between the automated tool and the manual queries, which validates the ALEAPP results used in section 5. The manual work also went further than the tool: it identified the active database, a deleted MMS and the true nature of five WhatsApp messages.

7.1 Automated verification (forensic_agent.py)

To make the manual procedure **repeatable**, it was automated in a Python script that uses only the standard library (hashlib, tarfile, sqlite3, csv). In one run it: (1) reads Data.tar once, computing MD5, SHA-1 and SHA-256 while extracting the target databases; (2) compares the hashes with every value listed in image_info.txt; (3) hashes the extracted copies, makes them read-only and creates working copies; (4) runs the SQL checks of section 6; (5) reads ALEAPP's TSV exports and compares the counts; (6) re-hashes everything; (7) writes a timestamped log, a JSON file and an HTML report. Run on Windows on 27 October 2026, it processed the 17.2 GB image in about two minutes.

Check	Result of the automated run
Integrity	MD5 aca33f3d…e7d9 and SHA-1 a7a4b184…73c4 match image_info.txt
SMS / MMS / threads	user_de: 0 / 0 / 0 (29 pages); data/data: 27 / 2 / 15 (31 pages); 0 free pages
Deletion trace	pdu: highest id 3, 2 rows → MMS 2 deleted; part: 5 of 5 rows present
Orphan MMS part	part 1, mid 2047483647 (placeholder), file app_parts/PART_1599870740204 (2020-09-12 00:32:20 UTC) — not found manually
WhatsApp	10 messages = ALEAPP; 3 locations + 2 live locations, 34.2 m apart, captions “Here I am!” / “Me, too!”; 0 revoked

Check	Result of the automated run
ALEAPP comparison	SMS 27 = 27, MMS 2 = 2, accounts 17 = 17, WhatsApp 10 = 10; 5 of ALEAPP's 6 empty WhatsApp rows matched to location messages by timestamp
Encrypted data	WhatsApp key file and encrypted backup msgstore.db.crypt12 both present in the image (backup decryptable)
Re-verification	Extracted copies IDENTICAL; only the working copy of msgstore.db-shm changed

The script contains no results: it recomputes them from the evidence. It reproduced every manual result, and it **corrected** one manual reading (part 1 is an orphan part, not a deleted row, section 6.5.1). Limitation: the files and checks are those learned in this case.

7.2 From a script to an agent (forensic_agent_v2.py)

A script follows a fixed list. An **agent** works in a loop: it **observes** the evidence, **decides** with rules, **acts**, and **records** each decision with its reason. Version 2 discovers every SQLite database by its signature (“SQLite format 3”) instead of using a list, and applies ten rules:

Rule	IF the agent observes…	THEN it…
R1	the hashes do not match the acquisition record	stops (the evidence cannot be trusted)
R2	two databases are byte-identical	analyses one copy (data/user/0 = data/data)
R3	a database has free pages	carves them for readable text
R4	a non-empty WAL file	lists text present only in the WAL
R5	numbering gaps (sqlite_sequence)	flags rows created then deleted
R6	a .db file without SQLite signature	flags a probably encrypted database
R7	the WhatsApp key file and .crypt12 backups	decrypts them (AES-GCM), checks the SQLite signature, compares
R8	a vault, root or encrypted-app folder	flags the application
R9	a known database (SMS, WhatsApp, accounts, Magisk)	runs the specialised analysis
R10	ALEAPP TSV exports	compares the counts

Run on the real image (Windows, 27 October 2026): 3,866 files extracted and re-verified IDENTICAL; 328 decisions; 32 findings.

Finding	Detail
Encrypted WhatsApp backups (R7)	**10 backups decrypted** with the key found in the image (data/data/com.whatsapp/files/key): the main backup and daily backups of 28 September to 5 October. All are valid SQLite databases. Backups up to 4 October contain 6 messages; those of 5 October contain 11, like the live database. No backup contains more messages than the live database (counts only; message-level comparison not done).
Probably encrypted databases (R6)	21 database files without SQLite signature, in Silent Phone, Firefox, Signal, Zoom and others — not readable without keys.
Free pages (R3)	Readable text in free pages of 5 databases (mainly Apple Music caches: httpcache.sqlitedb, MediaLibrary.sqlitedb).
WAL (R4)	26 text fragments present only in the WAL of Google Messages (bugle_db).
Privacy / root apps (R8)	11 flagged: HLD Vault, GalleryVault, Magisk, Signal, Wire, Silent Phone, Dust, ProtonMail, Tutanota, Tor Browser, Wickr.
Confirmed results	27 SMS, 2 MMS, 17 accounts, 10 WhatsApp messages = ALEAPP; MMS 2 deleted; 5 WhatsApp locations 34.2 m apart.

Lesson: fixed severity labels can mislead

v2 labelled numbering gaps in three WhatsApp tables (props, receipt_device, frequents) as HIGH deletions. These are internal tables (settings, delivery receipts, frequent contacts) that the application rewrites itself: the gaps are normal behaviour, not user deletions. This false positive led to version 3, where severity is computed from the content of each table.

7.3 A generic agent: the gap explorer (forensic_agent_v3.py)

Version 3 contains **no application knowledge** and accepts any extraction (.tar, .zip, folder) and any tool report (ALEAPP, iLEAPP…). It reproduces the method of this project automatically: (1) identifies every file by its signature and flags high-entropy database files as probably encrypted; (2) collects every path mentioned in the tool report and computes the **coverage gap** — the databases the tool never read; (3) samples every table for generic signals — timestamps (format recognised from the size of the number), message-like text, phone numbers, e-mail addresses, coordinates, URLs, blobs, numbering gaps, free pages, WAL text; (4) computes a score and a severity **from the data**; (5) proposes the next step with the tool to use and a ready-made SQL query for DB Browser. Every proposal must be verified by the examiner.

On a test image containing an invented messaging application unknown to any tool, it ranked that database first (HIGH): not read by the tool, timestamps in milliseconds, message text, phone numbers, GPS coordinates and two deleted rows. During the first runs on Windows it also met two real-world problems and now handles them: Android file names containing characters forbidden on Windows (“:”), stored under cleaned names with the original names kept in the records; and read errors of the USB disk, handled by re-opening the evidence and resuming at the same byte, the final hashes proving that no byte was lost.

Status and limits

Run on the real image (27 October 2026): 17,751 files extracted and re-verified IDENTICAL; 2,457 proposals produced, ranked by severity. Example: a Google account-metadata database not read by ALEAPP, with timestamps in milliseconds and URLs. The proposals have not all been reviewed: they are leads for the examiner, not findings. Limits: the agent knows which files the tool mentions, not every column it displays; its signals are heuristics that can produce false positives; it is a student prototype tested on one image. It assists the examiner and does not replace verification. Perspective: an AI planner choosing the next action under examiner control, with a local model and full logging.

8. Second tool: Autopsy (Windows) — processing and results

8.1 Purpose and set-up

Autopsy is a graphical forensic platform. It is used here as a **second, independent tool**. Because the PC's internal disk had only 22 GB free, the read-only .tar was processed directly from the external disk, with sleep and USB power saving disabled. Settings: time zone UTC; case `Android11_Pixel3` in `D:\Enquete_Android11\06_Autopsy`; data source type **Logical Files** (the evidence is a .tar file system, not a disk image).

8.2 Identifying the right file by its signature

The first attempt added the whole Extraction folder, and a second attempt selected a 4,096-byte file `._Android 11 - Pixel 3 - Data.tar`. Its first bytes, `00 05 16 07` followed by “Mac OS X” and “ATTR”, are the signature of a macOS **AppleDouble** metadata file, not of a tar archive. The correct file (17,205,267,456 bytes) starts with `64 61 74 61 2F` = “data/”, the name of the first entry of the tar header. The mistaken data source contains no phone data and is excluded from the analysis.

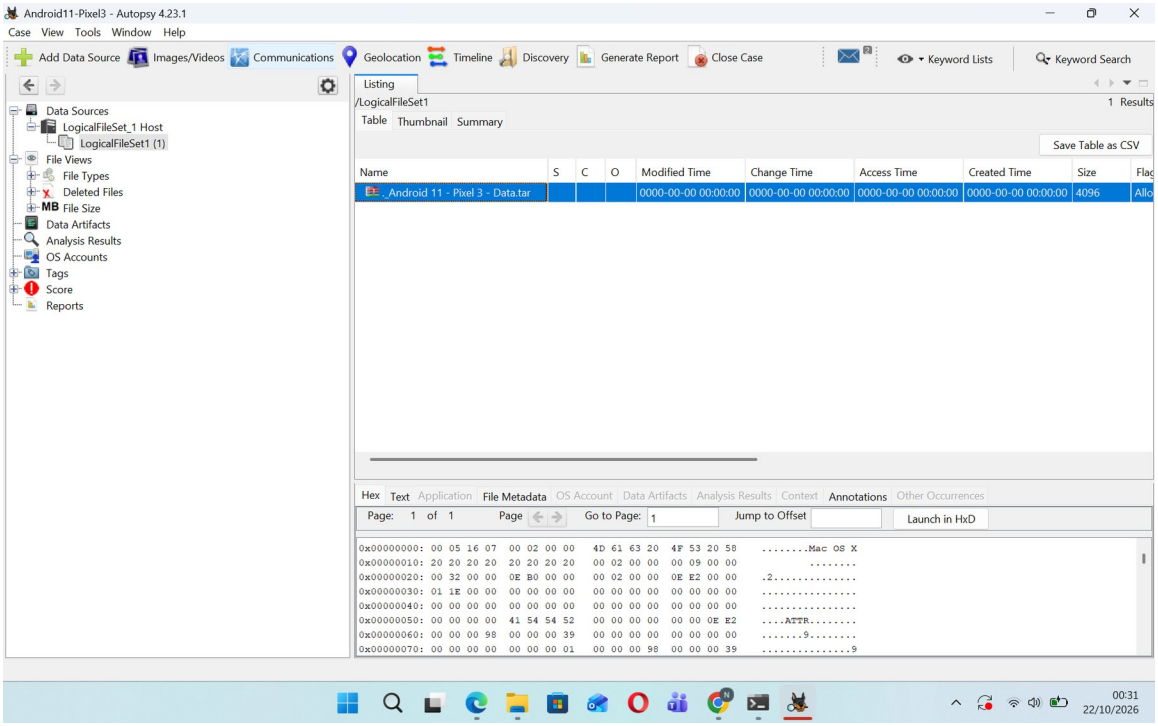


Figure 36. Autopsy hex view of the AppleDouble file (4,096 bytes): signature 00 05 16 07 and the text “Mac OS X … ATTR”.

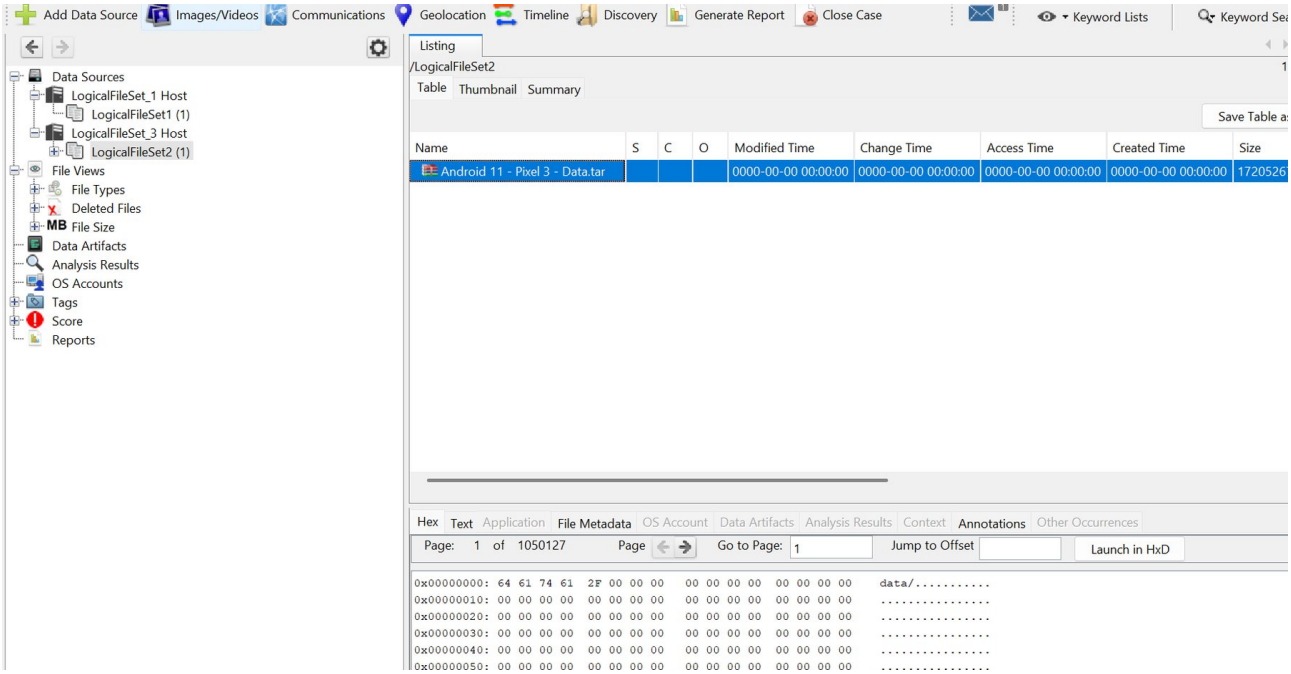


Figure 37. Autopsy: the real evidence (LogicalFileSet2), size 17205267456 bytes; the hex view begins with “data/”.

8.3 Processing results

Module	Duration	Result
Embedded File Extractor	1 h 35 min	Opened the tar; the folder data (46 sub-items) is visible under the data source
File Type Identification	56 min	Identified the extracted files by content
Android Analyzer	2 s	No artifacts produced (ran before the files were extracted)
Android Analyzer (aLEAPP)	≈ 0 s	Did not run on this data source (Autopsy reported 2 errors)

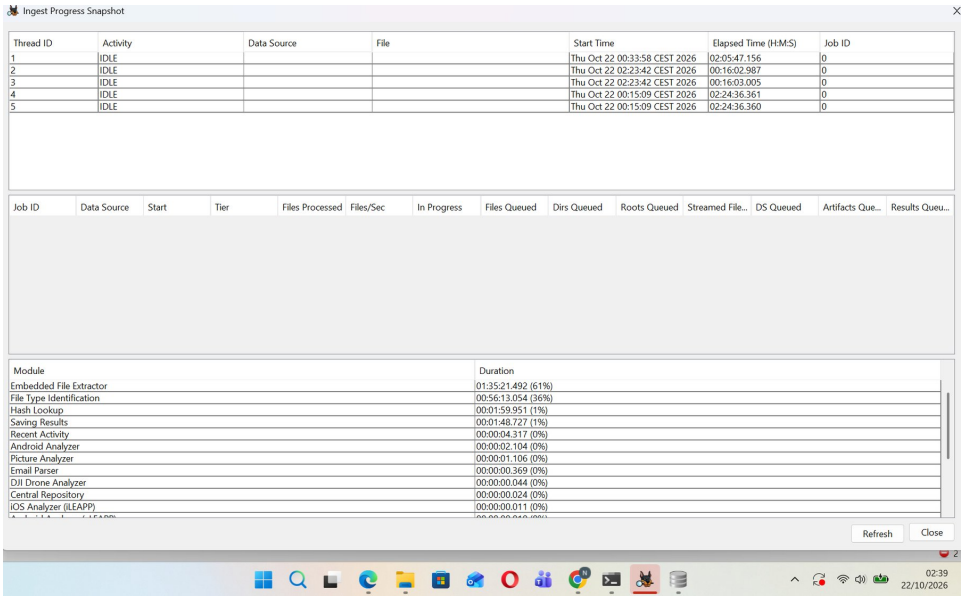


Figure 38. Autopsy Ingest Progress Snapshot at the end of the first processing: all threads idle; module durations.

8.4 Second run of the Android Analyzer

Because the Android Analyzer had run before the tar was extracted, it was run a second time on the extracted files (right-click LogicalFileSet2 → Run Ingest Modules → Android Analyzer only). This time it produced data artifacts:

Autopsy category	Count	Comparison with ALEAPP / DB Browser
Call Logs	27	ALEAPP lists 14 entries from calllog.db. The difference is not explained: possibly duplicates from data/data and its symbolic link data/user/0 (identical files, section 6.3), or additional call sources. To verify.
Communication Accounts	10 (1 device, 9 phone)	Phone numbers and the device account found in the communication artifacts.
Web History / Web Search	82 / 4	Browser history and search terms, consistent with section 5.8.
Web Accounts / Cookies / Bookmarks	489 / 834 / 2	Browser data.
Messages (SMS/MMS)	0 — category absent	ALEAPP and DB Browser found 27 SMS and 2 MMS. Probable cause: Autopsy read the empty mmssms.db in user_de (0 rows, section 6.4) rather than the active one in data/data. Hypothesis, not verified.

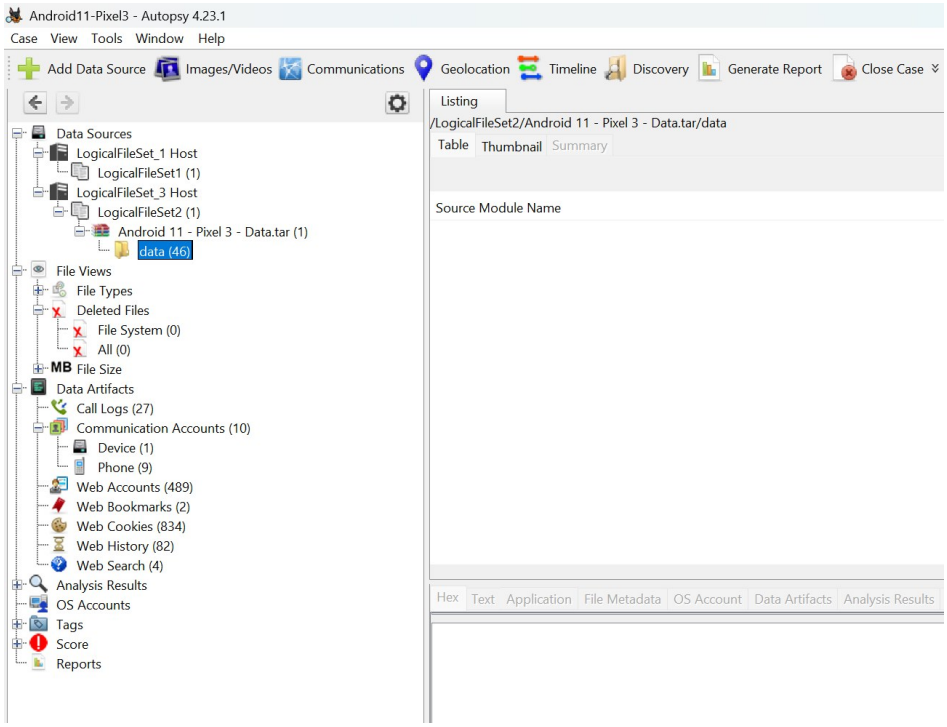


Figure 39. Autopsy after the second Android Analyzer run: Data Artifacts — Call Logs (27), Communication Accounts (10), Web Accounts (489), Web Bookmarks (2), Web Cookies (834), Web History (82), Web Search (4). No Messages category.

Result and limitation

Autopsy confirms, as an independent second tool, the **structure of the evidence** (the /data partition rebuilt from the tar, including the magisk_backup folder) and produces call, account and web artifacts. However, the result tables did not display (the Call Logs view showed “0 Results” and report generation listed no data source, with case errors logged), and **no SMS messages** were produced. The detailed validation of messages therefore relies on the manual SQLite verification (section 6), which shows why one tool must not be trusted alone.

9. Consolidated timeline

Local time = UTC − 4 h. Only events relevant to the investigation questions are listed. Source sections in the last column.

UTC	Local	Event	Sect.
2020-09-11 22:59:33	18:59	Factory reset (first boot after reset)	5.3
2020-09-11 23:08–23:11	19:08–19:11	Magisk installed; boot.img → magisk_patched.img → reboot	5.3
2020-09-11 23:11:48	19:11	First root grant to the ADB shell (device rooted)	5.3
2020-09-11 23:21:39	19:21	Google account thisisdfir@gmail.com added	5.2
2020-09-12 11:50–11:59	07:50–07:59	Batch installation of ~25 apps (vaults, encrypted messengers…)	5.8
2020-09-12 → 09-25	—	Registration SMS codes (Dust, imo, LINE, Signal, Telegram, TikTok, WhatsApp, Snapchat, Zoom)	5.4
2020-09-17 23:29–23:32	19:29–19:32	Images downloaded from Discord and Imgur (incl. BWV1Yuj.jpg)	5.5
2020-09-21 → 09-25	—	Kik, Wire, WhatsApp, imo, Viber chats and files with DFIR Two	5.4
2020-09-23 15:06:14	11:06	Screenshot_20200923-110614.png taken	5.6
2020-09-25 00:39:39	09-24 20:39	Same screenshot sent through imo	5.6

UTC	Local	Event	Sect.
2020-10-03 22:42–22:48	18:42–18:48	**WhatsApp location and live-location shares (“Here I am!” / “Me, too!”)**	6.6
2020-10-03 22:57:38	18:57	Location shared by DFIR Two via Skype	5.7
2020-10-03 23:59 → 10-04 00:02	19:59–20:02	Maps trip to The Club at 12 Oaks	5.7
2020-10-04 01:45–02:37	10-03 21:45–22:37	Duo and phone calls with DFIR Two	5.4
2020-10-04 01:57–02:06	10-03 21:57–22:06	Google Drive documents 12–14 created; document 12 trashed	5.5
2020-10-04 02:19–02:32	10-03 22:19–22:32	ProtonMail “For Android 11” + reply with JPG; Tutanota image downloaded	5.5
2020-10-04 09:26:58	05:26	Author's Mac mini authorised for ADB	5.3
2020-10-04 13:05:57	09:05	BWV1Yuj.jpg hidden in GalleryVault and uploaded to its Drive storage	5.6
2020-10-04 14:30–14:37	10:30–10:37	SMS/MMS exchange with DFIR Two (image sent, “Received.”, image received)	5.4, 6.5
2020-10-04 20:24:37	16:24	Screenshot hidden in HLD Vault	5.6
2020-10-04 20:55:11	16:55	Guest profile created	5.2
2020-10-05 19:29:15	15:29	Photo received by Nearby Share	5.5
2020-10-05 19:32:02	15:32	Acquisition computer authorised for ADB	5.3
2020-10-05 19:32:46–19:48:18	15:32–15:48	Root commands of the acquisition tool	5.3
2020-10-05 19:47:57–19:56:41	15:47–15:56	Imaging by Magnet ACQUIRE	4.2

10. Conclusions, answers to the questions and limitations

10.1 Answers

Q	Answer
Q1	Google Pixel 3, serial 8CEX1N716, Android 11; owner thisisdfir@gmail.com, +1 919-579-4674 (Google Fi); time zone UTC−4; factory-reset on 2020-09-11 and rooted with Magisk 12 minutes later (MagiskHide on).
Q2	Mainly “This Is DFIR Two” (+1 919-888-7386) over about twelve channels (SMS/MMS, calls, Duo, WhatsApp, Wire, Viber, Kik, Telegram, Houseparty, Dust, ProtonMail, Skype).
Q3	Yes: files entered (Discord, Imgur, Tutanota, Nearby Share) and left (MMS, Kik, Viber, imo, WhatsApp, ProtonMail, cloud: Google Drive, Google Photos, GalleryVault). Channels proven; confidential content not demonstrated.
Q4	Yes: two vault apps used on 2020-10-04, root hidden, encrypted/ephemeral messengers, a trashed Drive document, a deleted MMS row.
Q5	Holly Springs, Wake County, North Carolina; on 2020-10-03 evening the user and DFIR Two exchanged positions (WhatsApp, Skype) and the user drove to The Club at 12 Oaks.
Q6	Batch installation of privacy and messaging apps on 2020-09-12; explicit Play searches for ProtonMail and Tutanota; news/sport/Star Wars browsing on 2020-10-04.
Q7	See the consolidated timeline (section 9).

10.2 Overall conclusion

The evidence was verified against its 2020 acquisition record (identical MD5 and SHA-1, on two operating systems) and examined without alteration. The automated analysis with ALEAPP identified the device, its owner, a dense communication network centred on one correspondent, numerous file transfers, deliberate concealment and a coherent location and timeline. The manual SQLite verification confirmed every value checked and added findings the tool had missed (a deleted MMS, five WhatsApp location shares). **Deleted messages leave traces:** the numbering of the MMS table proves that one MMS was deleted, but its content is not recoverable because the database has no free pages and an empty journal; no WhatsApp message was deleted “for everyone”. Within the scenario, the **communication and exfiltration channels** are demonstrated; the **confidential nature** of the transferred files is not.

10.3 Limitations

- **File-system extraction:** no unallocated space, so deleted files cannot be carved; deleted records can only be found inside databases, WAL files or cloud-synchronised data (0 free pages in the databases checked).
- **Encryption:** Signal, Zoom, Silent Phone, parts of Telegram and 21 database files in total are unreadable without keys. The WhatsApp backups could be decrypted because their key was in the image (section 7.2).
- **Time interpretation:** some values are stored in local time, others in UTC; conversions rely on the demonstrated −4 h offset.
- **Cloud history:** some data older than the factory reset comes from server-side synchronisation, not from use of this phone at that time.
- **Second tool partial:** Autopsy produced call, account and web artifacts but no SMS, and its result tables did not display (section 8).
- **Test data:** the phone was populated by its author for research; no intent can be attributed to a real person.
- **Not yet done:** comparison with the author's documentation, hash comparison of the MMS image and uHJlBib.jpg.

Appendix A — Glossary

Term	Definition
Artifact	A trace recorded automatically by the system or an app as a side-effect of an activity (database row, log line, preference).
Acquisition (logical / file system / physical)	Creating a copy of device data: exported data only / all files / bit-for-bit copy of the memory.
ADB	Android Debug Bridge: lets a computer control the phone over USB; commands run as user shell (UID 2000); the computer's key must be authorised.
Root, UID 0, su	Super-user rights. su switches to root. Each app has its own UID; root has all rights.
Magisk / MagiskHide	Rooting tool that patches the boot image; MagiskHide hides root from apps.
FBE, CE, DE	File-Based Encryption. CE storage: readable after PIN unlock. DE storage: readable from boot.
Symbolic link	A path that points to another path; /data/user/0 points to /data/data.
SQLite	Single-file database used by Android apps; tables of rows and columns, queried with SQL.
WAL (-wal), -shm, -journal	SQLite companion files: recent changes not yet merged / index of the WAL / rollback journal.
Freelist / free pages	Pages released after deletions; they may still contain old data until reused.
sqlite_sequence	Internal table storing the highest AUTOINCREMENT number ever given for each table.
AppleDouble (._ files)	Hidden metadata files written by macOS on non-Apple file systems such as exFAT.
Signature (magic bytes)	First bytes that identify a file type regardless of its name (tar: entry name; AppleDouble: 00 05 16 07).
Hash (MD5, SHA-1, SHA-256)	Fixed-length fingerprint of a file; any change gives a different value.
Chain of custody	Record of who handled the evidence, when, how, and with which hash.
UTC, Unix time	UTC: universal reference time. Unix time: seconds (or milliseconds) since 1970-01-01 00:00:00 UTC.
Exfiltration	Unauthorised transfer of data out of an organisation.